

mlserve-ops

A machine-learning service, and everything around it — explained in plain words, from the very beginning.

What this document is. A guided tour of a real, working software project. It explains **what every piece is**, **why it had to exist**, and **where you will meet the same idea again** in your own work later.

Nothing here assumes you already know the words. When a new word shows up, it gets explained on the spot. If you can read a story, you can read this.

Machine learning

Web services

Benchmarking

Statistics

Docker

Monitoring

Ansible

Terraform

CI/CD

3

ways to run the same
model

255

automatic checks

64

inputs watched for drift

9

containers in the stack

0 £

cost to run

What is in here, and how to read it

Ten minutes of reading here will save you hours later. This page explains the shape of the whole document, and the one habit that makes it easy.

The one habit

Every single part of this document answers **four questions** in the same order. Once you notice the pattern, you can skim any part and still get what matters:

QUESTION	WHAT IT MEANS
What is it?	The plain-English name and a picture of the thing. No jargon on the first line, ever.
Why did we need it?	The problem that existed <i>before</i> it. This is the important one — tools only make sense as answers to problems.
How does it work here?	The specific details in this project, including the real file names and real numbers.
Where will you use this again?	Other jobs, other projects, other problems the same idea solves. This is the part that transfers.

Imagine it like this

Someone hands you a bicycle and says "this is a bicycle." That teaches you almost nothing. But if they say "before bicycles, people walked everywhere and it took all day to visit a friend; a bicycle fixes that" — now you understand bicycles, and you will also understand motorbikes, cars and trains. Tools are answers. Learn the question and you have learned the tool.

Contents

- 1 What is in here, and how to read it**
The habit, and the map
- 2 The whole story in one page**
What this project is, in a single breath
- 3 Sixteen words you need first**
The vocabulary, in plain language
- 4 The brain: a machine that reads digits**
Neural networks, training, and why we picked a tiny one
- 5 Three copies of the brain: the runtimes**
TorchScript, ONNX, and int8 — and what quantisation actually does
- 6 The shop counter: the API**
How other programs ask your brain a question

7 The race with a stopwatch: benchmarking
Measuring speed honestly, which is much harder than it sounds

8 How to avoid fooling yourself: the statistics
Confidence intervals, noise floors, and the most honest result here

9 The smoke alarm: drift detection
Noticing when the world stops looking like the training data

10 The photocopy you actually test: backup
Why a backup you have never restored is not a backup

11 The settings drawer, and the remote control
Config files and `tasks.py`

12 The lunchbox: Docker
Making software run the same on every computer

13 The whole kitchen: Docker Compose
Nine containers that start in the right order

14 The dashboard and the alarm bell: monitoring
Prometheus, Grafana, Alertmanager

15 The robot installer: Ansible
Setting up a hundred servers without doing it by hand

16 The robot builder: Terraform
Creating computers by writing down what you want

17 The homework checker: CI/CD
Robots that test your work before anyone sees it

18 How we know it works: the tests
255 checks, and the ones that matter most

19 The whole picture
Every piece on one diagram, and how a request flows through

20 Twenty-one mistakes, and what each one teaches
The most valuable part of this document

21 Where you will use each of these things
A table for every tool: what it is for, in your future jobs

22 Glossary and command cheat sheet
Every word, and every command, in one place

WHERE WILL YOU USE THIS AGAIN?

The four-question habit is not about this project. It is how you read *any* unfamiliar codebase, *any* tool you are handed at a new job, and *any* technology that did not exist last year. "What problem did this exist to solve?" is the single most useful question in software.

The whole story in one page

If you only read one page of this document, read this one.

The problem

Imagine you have taught a computer to recognise handwritten numbers. It looks at a small picture of a digit and says "that is a 7." Great. But a brain sitting in a file on your laptop helps nobody.

To be *useful*, that brain has to become a **service** — something other programs can ask questions of, over the internet, all day and all night. And the moment you do that, a flood of new problems arrives, none of which are about machine learning at all:

Which version is faster?

There are several ways to run the same brain. Which one should we pay for in production? Guessing is not good enough.

Is it still correct?

If we swap to a smaller, faster version, does it still get the right answers? Someone has to prove it, automatically.

What if the world changes?

The brain learned from old pictures. What if new pictures look different? Nobody notices until the results are quietly wrong.

What if it breaks at 3am?

Someone needs to be told. And someone needs to know the data can be brought back.

The answer

This project is a small, complete, honest machine-learning service — plus every piece of engineering that a real one needs around it, and a benchmark harness that is allowed to tell you the truth, *including when the truth is "this measurement is not good enough to conclude anything."*

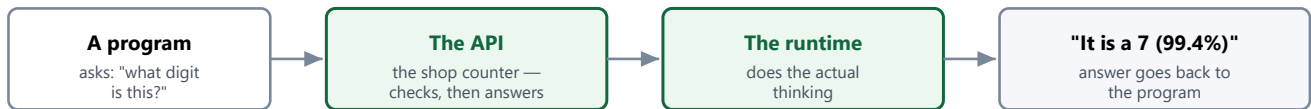
That last part is what makes it unusual. Most benchmarks are built to produce an impressive number. This one is built to produce a *defensible* one — and when the data does not support a conclusion, it says so out loud instead of picking the flattering reading.

How the pieces fit together

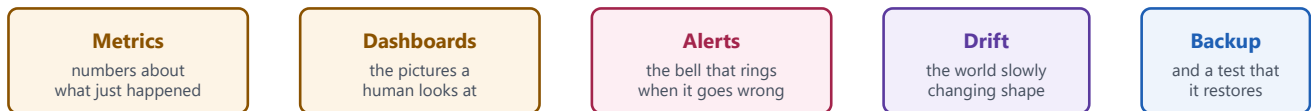
1 · BUILD THE BRAIN (happens once, or once per release)



2 · SERVE IT (happens every single second)



3 · WATCH IT, AND BE READY WHEN IT BREAKS



Three jobs, three rows. Building happens rarely; serving happens constantly; watching happens quietly in the background.

The surprising ending

You might expect the story to finish with "and int8 was 3× faster, so we shipped it." It does not. It finishes like this:

THE HONEST RESULT

The int8 copy of the brain really is **3.5× smaller** (151 KB instead of 535 KB) and just as accurate — those are solid facts. But whether it is *faster* at answering, this harness cannot say. The run-to-run variation on this machine, measured against itself, is a band from **0.58× to 1.71×**. Every speedup measured sits inside that band.

So the conclusion is: we proved the size and accuracy wins, and we proved we do *not* have the evidence to claim a speed win. A weaker project would have reported the biggest number it saw and moved on.

WHERE WILL YOU USE THIS AGAIN?

Everywhere performance is discussed. "It is 40% faster" is one of the most common claims in software and one of the least often justified. Learning to ask "compared to what, measured how many times, and how much does the machine vary against itself?" will change how you read every benchmark you ever see — including the ones from companies selling you something.

Sixteen words you need first

Every word below appears later in this document. Read them once now, and nothing further will confuse you. You do not need to memorise them — they are all repeated in the glossary at the end.

Words about the brain

WORD	WHAT IT MEANS, PLAINLY
Model	A computer program that has <i>learned</i> from examples instead of being told the rules. Here: one that recognises handwritten digits. Also called "the brain" in this document.
Training	Showing the model thousands of examples with the right answers attached, and letting it adjust itself until it gets them right. Like practising with a workbook that has the answers in the back.
Neural network	The particular style of model used here. It is a stack of very simple maths steps, each passing its result to the next. Nothing about it is like a real brain, despite the name.
Weights	The numbers inside the model that training adjusts. Training's entire output is a big pile of weights. Here: 136,586 of them.
Parameters	Another word for those same numbers. "A 136k-parameter model" means "a model with 136,000 adjustable numbers."
Inference	Using a trained model to answer a question. Training is the studying; inference is the exam. This document mostly cares about inference, because that is what production does millions of times.
Accuracy	The share of questions it got right. 0.9741 means 97.41 times out of 100.

Words about shipping it

WORD	WHAT IT MEANS, PLAINLY
Runtime	The program that <i>runs</i> the model — the engine, not the fuel. Different engines can run the same weights at different speeds.
API	Application Programming Interface. A fixed set of questions that one program agrees to answer for another. Like a restaurant menu: you do not walk into the kitchen, you order from the list.
Endpoint	One specific item on that menu — one address that does one thing. <code>/predict</code> is an endpoint.
Latency	How long one request takes, in milliseconds. Lower is better. The number everyone argues about.
Throughput	How many requests you can handle per second. Related to latency but not the same thing — like how a shop can be slow per customer but serve many customers at once.

WORD	WHAT IT MEANS, PLAINLY
p50 / p95 / p99	Percentiles. p50 is the middle request ("half are faster than this"). p99 is the slow one in a hundred. p99 matters most, because a slow request is often a broken one. Also the hardest to measure — see Part 7.
Batch	A group of questions sent together in one request. Usually faster per question than sending them one at a time, because some costs are paid once per batch rather than once per question.

Words about trusting the numbers

WORD	WHAT IT MEANS, PLAINLY
Reproducible	Run it again, get the same answer. This is the single most important property of a benchmark and the easiest to lose by accident.
Confidence interval	A range that probably contains the true value. " $1.23 \times [1.15, 1.31]$ " means the measured speedup is 1.23 and we are fairly sure the real one is between 1.15 and 1.31.
Noise	Random variation that has nothing to do with what you are measuring — a background program waking up, the processor getting warm. The enemy of Part 7.
Drift	When the real world slowly stops resembling the examples the model learned from, so its answers quietly get worse. <i>Drift</i> as a verb: the input distribution has drifted.

WHY WE NEEDED THIS PAGE

Technical writing usually fails at exactly one place: it uses a word the reader has never seen, on the assumption that the reader has seen it. Nobody admits confusion at that point; they just stop reading. Sixteen words is a small price for the rest of the document being readable.

WHERE WILL YOU USE THIS AGAIN?

These sixteen words are the shared vocabulary of the entire industry. They appear in job advertisements, code reviews and interviews regardless of language, company or framework. Knowing them precisely — particularly the difference between latency and throughput, and between p50 and p99 — is the difference between sounding like you have used these tools and actually having used them.

The brain: a machine that reads digits

Everything else in this project exists to serve this one small thing. So it is worth understanding it properly — and worth understanding why we deliberately made it small.

What is it?

A neural network that looks at a tiny 8×8 picture of a handwritten digit and says which digit it is, from 0 to 9. The picture is only 64 pixels — about the size of the icons on an old phone screen.

8×8

pixels per picture

1,797pictures in the
dataset**136,586**

adjustable numbers

97.41%correct on unseen
pictures**~4 s**

to train on a laptop

Why did we need it?

THE PROBLEM BEFORE IT

This project is not really about digit recognition. It is about the *machinery around* a model — how you serve it, measure it, watch it and protect it. But you cannot demonstrate machinery without something to put inside it.

So we needed **a real model** — one that genuinely trains and genuinely predicts — that was **also small enough** to make every other part of the project practical. Those two requirements pull against each other, and the whole design of the model is the compromise between them.

Why not use a big impressive model?

This is the most important design decision in the project, so here is the reasoning spelled out.

IF WE USED A LARGE MODEL	WHAT WOULD BREAK
Training takes hours on a GPU	Nobody can run it to check your work. Every claim becomes "trust me, it worked on my machine."
It costs money to run	The project's rule is zero cost . A free-tier machine cannot train a large model.
It needs to download gigabytes	The benchmark would be measuring your internet connection on the first run and nothing on later runs.
The model file is hundreds of megabytes	Comparing "535 KB vs 151 KB" would be a rounding error lost in the noise. The size experiment would prove nothing.

IF WE USED A LARGE MODEL	WHAT WOULD BREAK
Timing differences get large	Sounds good — but then the benchmark's careful statistics have nothing interesting to say. The interesting lesson is precisely that <i>small differences are hard to measure</i> .

The model is small on purpose. The model is not the subject of this project; the rigour around it is.

How does it work here?

The data

The pictures come from `sklearn.datasets.load_digits`, which is bundled inside an already-installed library. Two consequences, both deliberate:

- **No download.** The benchmark works with the network switched off, on a plane, or inside a locked-down build machine that is not allowed to fetch anything.
- **Identical for everyone.** If the data were downloaded, two people could silently get different versions and their results would not be comparable. Here, "run it again and get the same answer" is true by construction.

Splitting the data — and one mistake that ruins everything

The 1,797 pictures are divided into three groups:

GROUP	SHARE	WHAT IT IS FOR
Train	70%	The workbook with answers. The model learns from these.
Validation	15%	A mock exam, used while tuning. The model never learns from these directly.
Test	15%	The real exam, opened once at the end. This is where "97.41% accurate" comes from.

THE MISTAKE THIS AVOIDS — DATA LEAKAGE

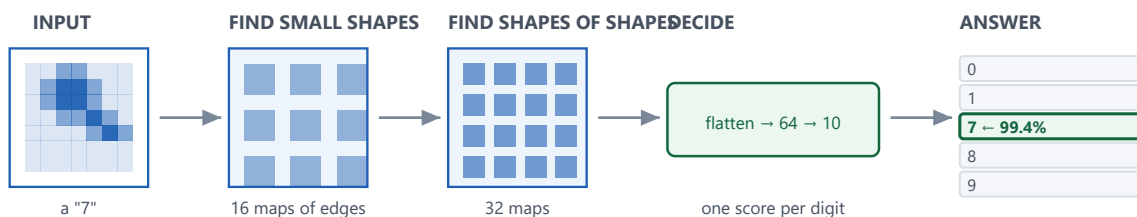
Suppose you wanted to make the numbers look good. You could let the model see the test pictures during training. It would then score ~100% on the test — and be useless in the real world, because in the real world it will never see those pictures again.

This is called **leakage**, and it is the single most common way machine-learning results become fiction. It is rarely done on purpose. It happens by accident, and it is invisible in the output — the numbers just look slightly too good.

There is a subtle version of it in this project. The pictures are normalised (rescaled) by dividing by 16. If that rescaling had been computed over *all* the pictures including the test ones, the test data would have influenced the model's inputs — a tiny leak, and a leak all the same. In `src/mlserve/model.py` the statistics come from the **training split only**, and the code comments say why.

The network itself

The architecture is two "convolution" layers followed by a small decision layer. Convolution is the trick that made image recognition work: instead of looking at all 64 pixels at once, the network slides a small 3×3 window across the picture looking for tiny local patterns — an edge, a corner, a stroke. Then it looks for patterns among those patterns.



The whole model: find small shapes, find shapes made of those shapes, then decide. Four layers, 136,586 numbers.

Why the model is shaped this way

CHOICE	REASON
Two convolution layers, not twenty	It has to be simple enough that the exported ONNX file is readable when opened in a graph viewer. A 50-layer network produces a diagram nobody can check.
Padding of 1 on the 3×3 windows	Keeps the picture 8×8 throughout, so the arithmetic is easy to follow. It also means the flatten step has a predictable size.
Plain code, no clever tricks	<code>torch.jit.script</code> (Part 4) compiles this to a standalone file. It refuses to compile unusual or highly dynamic code. Writing boring code here makes the export work.
Dropout of 0.1	Randomly ignores 10% of connections during training, which stops the model memorising the training pictures. This is the standard guard against overfitting — being good at the workbook and bad at the exam.

CHOICE	REASON
Seed 1337 everywhere	Computers generate "random" numbers from a starting value. Fix that value and the randomness repeats exactly. It is the difference between a rerun and a different run.

About the seed — a small idea with big consequences

WHY WE NEEDED A SEED

Training involves randomness: the order pictures are shown, the initial weights, the dropout mask. Without a fixed starting point, running training twice gives two slightly different models — and then two slightly different accuracies, and no way to tell whether a change you made mattered or whether you just got a luckier shuffle.

Fixing the seed makes every run identical. It is the foundation of the entire reproducibility claim: **the same command produces the same numbers, on your machine and mine.**

A TRAP WE HIT: THE SEED THAT WAS NOT BEING SET

A subtler version of this bit us in the benchmark. Python deliberately randomises certain internal hash values *every time it starts* — a security feature, so that an attacker cannot predict them. Our benchmark was using those values to decide which pictures to time.

The result: each run timed a *different sequence of pictures*, so two runs of the identical command were not comparable, and the difference was indistinguishable from ordinary timing noise. It looked like normal measurement wobble. It was a bug.

The fix: replace the built-in hashing with SHA-256, which is identical in every process and every version of Python. See Part 21 for the full story.

WHERE WILL YOU USE THIS AGAIN?

- **Train/test splitting and leakage** apply to every model you ever build, in any framework. The accidental version — computing a scaling factor or a filter over all your data — is the one that catches people.
- **Random seeds** matter in any reproducible work: simulations, tests that involve randomness, data splitting, anywhere a result depends on a shuffle.
- **Deliberately choosing a small model** to make everything else testable is a professional judgement, not a shortcut. Being able to explain *why* the small model is the right choice is what distinguishes engineering from showing off.
- **Convolution** — looking for small local patterns and combining them — is the idea behind virtually all image and audio models in use today.

Three copies of the brain: the runtimes

The same trained model, packaged three different ways. This is the experiment the whole project is built around — so it matters that the three copies really are the same brain.

What is it?

A model's weights are just numbers. But numbers need a *program* to interpret them, and there is more than one program that can do that. It is like music: the same song can be on a vinyl record, a CD or a streaming file. Same music, different containers, different trade-offs in quality and convenience.

COPY	WHAT IT IS	PLAIN DESCRIPTION
TorchScript <code>model.ts</code>	The original framework, frozen	PyTorch is the library the model was written in. TorchScript takes the model and <i>freezes</i> it into a standalone file that no longer needs your Python code. The originals, essentially unchanged.
ONNX fp32 <code>model.onnx</code>	A universal interchange format	ONNX is a common language that many tools speak. Exporting to it means the model can run on engines that have never heard of PyTorch. Same full-precision numbers.
ONNX int8 <code>model.int8.onnx</code>	The compressed version	Every number in the model is stored in a smaller box. Almost a third of the size. Slightly less precise by construction — the question is whether the lost precision matters.

Why did we need it?

THE PROBLEM BEFORE IT

You trained a model in PyTorch. Now you have to run it in production, a million times a day. Do you ship PyTorch? It is enormous, it is slow to start, and it drags in a whole ecosystem of dependencies you do not need.

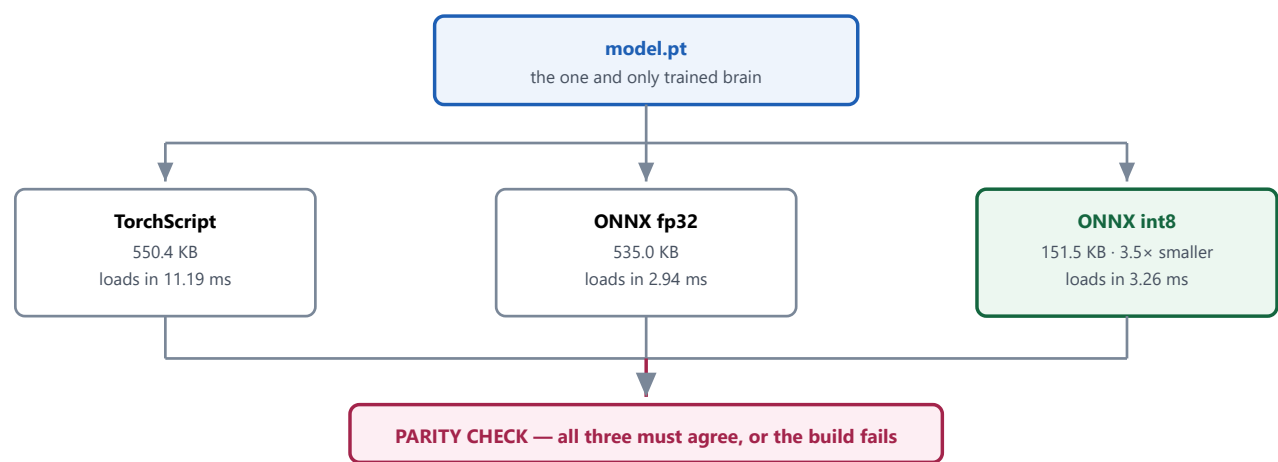
There are alternatives. Which one is fastest? Which one is smallest? Does the smallest one still get the right answers?

These are ordinary engineering questions with measurable answers — but only if you can compare fairly. That is what this part of the project is.

The most important rule: one brain, three boxes

Here is the trap. Suppose you train three separate models, export each one differently, and compare their speeds. You would learn nothing, because the three models might genuinely be different — different weights, different accuracy. The speed difference could be caused by the model, not the runtime.

So all three copies are exported from **exactly the same trained weights file**. The only thing that differs between them is the container. Any difference that shows up is caused by the runtime, and nothing else.



One brain exported three ways, then checked against each other. This is what makes the speed comparison meaningful.

How does it work here?

What int8 actually means

This is worth understanding properly, because "quantisation" is one of those words that gets used a lot and explained rarely.

Imagine it like this

You measured your height really carefully: 1.4273 metres. Someone writes it down as "about 1.4 metres."

You lost some detail. But you also used far fewer characters, and for most purposes "about 1.4 metres" is completely fine. Quantisation is exactly this: writing the numbers down with less detail, in exchange for taking up much less space.

And crucially — if the question is "am I taller than the door frame?", both answers are identical. The lost detail did not change the answer.

Technically: normal model numbers are 32-bit floats, which store a number with a decimal point and a lot of precision. int8 stores each number as an integer from -128 to 127 — one byte instead of four. A quarter of the space, and smaller numbers are quicker for a processor to move around.

TYPE	HOW STORED	SPACE EACH	ANALOGY
fp32	Decimal, high precision	4 bytes	1.4273 metres
int8	Whole number, -128...127	1 byte	about 1.4 metres

The result, and a limitation we admit openly

3.5×

smaller file

97.41%

accuracy — identical

0.0%

disagreement with fp32

?

faster? unproven

So the compression worked perfectly: a third of the size, zero loss in accuracy, zero disagreement with the full-precision version on the test set. Those are excellent, solid results.

THE HONEST LIMITATION

Our int8 model is **not fully compressed**. Only the "dense" parts of the network were converted. The convolution layers — the image-scanning part — had to stay in full precision, because of a bug described in Part 21.

A proper static-quantisation pipeline would compress those too, and get a smaller file than 151 KB. We recorded this in the artifact's metadata and stated it in the report, rather than presenting 3.5× as the best achievable.

Why admit it? Because someone reading "3.5× smaller" might reasonably assume that is the limit of the technique, and go on to make a bad decision later. Naming your own ceiling is part of reporting a result honestly.

The parity check: turning agreement into a gate

After exporting, the code runs all three copies over the same 270 exam pictures and compares their answers:

RUNTIME	TOLERANCE FOR DISAGREEMENT	WHY
TorchScript	0%	It is the reference. Nothing to compare against.
ONNX fp32	0%	Same numbers in a different format — it should be exactly identical.
ONNX int8	up to 2%	Compression loses detail by design. A little disagreement is expected and acceptable.

If any copy exceeds its tolerance, the export **fails the build**. Not a warning printed and ignored — an actual failure that stops everything.

WHY FAILING THE BUILD MATTERS SO MUCH HERE

A broken export is the worst possible bug for this project, because it produces plausible-looking numbers. If the int8 model were quietly wrong, the benchmark would happily report "int8 is fast!" while it was actually just guessing. Every subsequent conclusion would be built on sand.

Warnings get ignored. A build failure cannot be. So the check is a gate, not a note.

WHERE WILL YOU USE THIS AGAIN?

- **Model compression is standard practice** for putting models on phones, in browsers, on cheap hardware, or anywhere memory is expensive. The same trade-off — size and speed against precision — appears every time.
- **ONNX specifically** is how models move between tools. "I trained it in PyTorch but need it in a C++ program" is a solved problem, and ONNX is usually the answer.
- **Parity testing** — proving that a rewritten, optimised or compressed version of something behaves identically — is a general technique. It applies to database migrations, language rewrites, and compiler changes, not just models.
- **Making a check fail the build rather than printing a warning** is a habit worth adopting everywhere. It is the difference between a check that exists and a check that works.

The shop counter: the API

The model is a brain in a jar. The API is how the outside world talks to it. This part explains what a web service actually is, and the small decisions that separate a service you can trust from one you cannot.

What is it?

A program that sits and waits for other programs to ask it questions over the network. It listens at an address, understands a few specific requests, and sends back answers.

Imagine it like this

*A restaurant. You do not walk into the kitchen and start cooking. You sit at a table, **order from a menu**, and a waiter takes your order to the kitchen and brings back your food.*

The menu is the API. It lists exactly what you are allowed to ask for and how to ask for it. The waiter is the code that receives your request and passes it along. And a good waiter does something else important: if you order something not on the menu, they tell you clearly, rather than bringing you a random plate of food and hoping.

Why did we need it?

THE PROBLEM BEFORE IT

Without an API, using your model means: install Python, install the right libraries, write code that loads your model file, and call it. That only works for people who are programmers, on machines where all of that is installed.

With an API, *anything* that can make a network request can use your model. A phone app, a spreadsheet, a tiny script in any language, a website. The model becomes a **service** rather than a file.

How does it work here?

The endpoints — the menu

ADDRESS	TYPE	WHAT IT DOES
<code>/healthz</code>	liveness	"Am I alive?" Answers instantly. Deliberately checks nothing else.
<code>/readyz</code>	readiness	"Can I actually serve?" Checks the model really loads. Fails if not.
<code>/metrics</code>	monitoring	Machine-readable numbers about what the service has been doing.
<code>/models</code>	inventory	Lists the three runtimes: present? loaded? how big? how slow to load?

ADDRESS	TYPE	WHAT IT DOES
<code>/stats</code>	inventory	How many requests have been saved to the log.
<code>/predict</code>	inference	One picture in, one answer out.
<code>/predict/batch</code>	inference	Up to 64 pictures in, 64 answers out.
<code>/docs</code>	documentation	A self-generated, clickable page describing every endpoint.

That last one is worth pausing on. `/docs` is generated automatically from the code itself, so it cannot fall out of date. If you open `http://localhost:8000/docs` you get a page where you can look at every endpoint, see exactly what it expects, and *click a button to try it*. Nobody wrote that page by hand.

Liveness versus readiness — the distinction that trips everyone up

This is one of the most commonly misunderstood ideas in running services, and getting it wrong causes outages. So here it is carefully.

SIGNAL	QUESTION	WHAT SHOULD HAPPEN IF IT FAILS
Liveness <code>/healthz</code>	"Is this process still running, or has it frozen?"	Restart it. A frozen process will never recover on its own.
Readiness <code>/readyz</code>	"Should I send it traffic right now?"	Do not restart it. Just stop sending requests until it is ready. It may be starting up, or waiting for something.

THE MISTAKE, AND WHY IT IS SO DAMAGING

Most people wire the model check into *liveness*. The reasoning seems sensible: "if the model is broken, the service is broken, so restart it."

Here is what actually happens. A bad release ships a broken model. Every copy of the service reports "not alive." The orchestrator dutifully restarts all of them. They come up, load the same broken model, and report "not alive" again. Restart. Restart. Restart.

You now have two problems. The original broken model — *and* a restart storm hammering your system, making logs unreadable and masking the real cause. The original error is buried under thousands of identical restart messages.

Keeping them separate means a broken model produces exactly one clear signal: "ready, but not ready to serve" — and the service just quietly sits out of rotation, logging one honest error. That is far easier to diagnose at 3am.

Things this API does that a naive one would not

Request logging happens *after* answering

Saving a row to the database is slow. If it happened before answering, the customer would wait for it — and worse, the benchmark would measure database speed and call it inference speed. So it is queued as a background task.

Oversized batches are rejected, not trimmed

Send 1,000 pictures, get an error saying the limit is 64. A friendlier-looking service might silently process the first 64 and return those answers — but then your program believes it has 1,000 results and quietly loses 936. A clear failure beats a silent wrong answer, always.

Errors return errors, not 500s

If you send nonsense, you get a 422 "your request was malformed" with a readable explanation — not a 500 "something exploded", which tells you nothing and suggests the server is at fault.

Confidence, not just an answer

Every prediction comes with a percentage. "It is a 7, and I am 99.4% sure" is far more useful than "7" — because a low-confidence answer is a signal that the input is unusual.

A worked example

```
# Ask the service about one 8x8 picture, using the int8 runtime
curl -X POST "http://localhost:8000/predict?runtime=onnx-int8" \
  -H "Content-Type: application/json" \
  -d '{"features": [0,0,5,13,9,1,0,0, 0,0,13,15,10,15,5,0, ... ]}'

# Answer:
{
  "predictions": [
    {
      "predicted_class": 7,
      "confidence": 0.994,
      "probabilities": [0.001, 0.000, ... ]
    }
  ],
  "runtime": "onnx-int8",
  "batch_size": 1,
  "latency_ms": 3.41
}
```

Note the `?runtime=` in the address. That is how the benchmark tests all three runtimes through the exact same service — one small switch, so that everything else about the request stays identical.

WHERE WILL YOU USE THIS AGAIN?

- **Liveness versus readiness** is used by Kubernetes, Docker Swarm, AWS load balancers, Azure, and every orchestrator. Getting this distinction right is a standard expectation in DevOps and platform roles.
- **Background tasks for slow side-effects** — logging, sending emails, updating a cache — appear in every web service you will ever work on.
- **Rejecting bad input loudly** instead of silently adjusting it is a principle that applies to every interface you design, including command-line tools and internal function calls.
- **Auto-generated API documentation** (this project uses FastAPI's built-in ability) is how modern teams keep documentation honest. If you have ever used a service whose docs were wrong, you know why this matters.

The race with a stopwatch: benchmarking

Measuring how fast software runs sounds like the easiest job in computing. It is one of the hardest to do honestly. This part explains what the benchmark does, and every decision exists because a simpler version produced a wrong answer.

What is it?

A program that runs the model thousands of times in carefully controlled ways, records how long every single call took, and writes the results to a file so they can be analysed and re-checked.

48

configurations tested

1,344

timed calls per run

2

different methods compared

3

repeats of everything

Why did we need it?

THE PROBLEM BEFORE IT

You have three ways to run your model. You must pick one for production. Somebody will ask you why.

You could time one request of each on your laptop and pick the winner. But then: was your laptop busy? Was the first one penalised for warming up a cache? Was the difference real, or just one lucky run? Would it still be true tomorrow?

Without a real harness you have an opinion. With one you have evidence. And crucially, the evidence is *checkable by someone else* — which is the difference between a claim and a measurement.

How does it work here?

What gets measured

The benchmark runs every combination of these four things — $2 \times 3 \times 4 \times 2 = 48$ configurations:

SETTING	OPTIONS	WHY IT IS A VARIABLE
Mode	<code>inproc</code> , <code>http</code>	Do we time just the model, or the whole round trip through the API? Both, because they answer different questions.
Runtime	torchscript, onnx-fp32, onnx-int8	The main question: which is faster?
Batch size	1, 8, 32, 64	Does handling 64 at once beat handling them one at a time?

SETTING	OPTIONS	WHY IT IS A VARIABLE
Concurrency	1, 4	Does sending several requests at the same time help, or just make everything slower?

Two ways of measuring, and why both are needed

inproc — measure ONLY the model



http — measure the WHOLE round trip



Typical: 2 – 25 ms

Big enough that noise matters less — so this is the trustworthy mode

The same request, measured two ways. The difference between them is the API's own cost, and you can only see it by measuring both.

WHY WE NEEDED BOTH MODES

If you *only* measured `inproc`, you would know which engine is fastest but not what your customers experience. If you *only* measured `http`, a difference could come from the model or from the API layer — and you could not tell which.

Measuring both means a difference that appears in one mode but not the other can be **attributed**. That is the difference between "it is slower" and "the API layer is slower, the model is the same."

Reproducibility: the property everything else depends on

Five separate things are pinned so that a rerun is genuinely a rerun:

WHAT IS PINNED	WHAT IT PREVENTS
The random seed	Different pictures being timed each run.
Per-configuration seeds from SHA-256	Python's per-process random hashing, which silently made each run different. <i>This was a real bug.</i>
Thread counts fixed at 1	Results that depend on how many processor cores your machine happens to have.
Warm-up calls, discarded	Measuring the one-time cost of loading a model and calling it "inference speed."
The server started fresh by the benchmark	Results depending on a long-running process with different memory layout.

WARM-UP: THE EASIEST MISTAKE TO MAKE

The very first time you run a model, the runtime does a lot of one-time setup: building internal graph structures, allocating memory, deciding how to schedule the work. The second call reuses all of it.

If you include that first call in your average, you are mostly measuring start-up cost and calling it speed. So the benchmark runs warm-up calls first and **throws them away**. It measures the steady state, which is what production actually experiences after the first request.

One call is one measurement, no matter how big

A subtle detail with a big effect. When a batch of 64 pictures is sent in one request, that counts as **one** latency measurement — not 64.

Imagine it like this

You order 64 pizzas in one phone call. The call takes 3 minutes. That does not mean each pizza took 3 minutes! If you counted it that way, ordering in bulk would look 64 times slower than ordering one at a time — which is the exact opposite of the truth.

What the benchmark refuses to do

One decision is worth calling out because it removes a result the project would otherwise be able to show.

While benchmarking, request logging is **switched off**. Saving each request to the database is slow, and because of how the web server works, those saves would show up in the *next* request's timing. The benchmark would then be measuring database write speed and reporting it as inference speed.

So logging is off — and the consequence is that the cost of logging is not measured anywhere in this project. That is stated plainly in the report rather than left as an unmentioned gap.

WHERE WILL YOU USE THIS AGAIN?

- **Warm-up exclusion** applies to any performance test: database connection pools, web servers, caches, compilers. Anything with one-time setup cost.
- **Separating layers** — measuring the component versus the whole system — is how you find where time is actually going. It is the basis of all profiling.
- **Pinning thread counts and seeds** is essential for any result you intend to compare, publish, or defend. It is also a standard part of designing scientific experiments.
- **Stating what you did not measure** is a professional habit that builds trust far faster than claiming everything was covered.

How to avoid fooling yourself: the statistics

This is the most important part of the document, and the one most projects skip. It contains the single most valuable lesson in the whole repository.

What is it?

A small set of mathematical tools that turn raw timings into statements you can defend — and that tell you when the data simply does not support a conclusion.

Why did we need it?

THE PROBLEM BEFORE IT

The benchmark produces 1,344 numbers. Some average is "faster" than another. Is that a real difference, or is it just the machine being slightly different from one moment to the next?

Here is the thing that makes this genuinely hard: **there is no way to tell by looking at the numbers**. A 15% speedup caused by a real improvement and a 15% speedup caused by your laptop's fan spinning up look identical in the output.

You need a way to measure your own measurement error. That is what this part is about.

The story of the flip

This is the moment the project changed direction, and it is worth telling properly.

We ran the benchmark twice. Identical command, identical machine, identical everything. And the results **contradicted each other**:

RUN	CLAIM ABOUT INT8 AT BATCH 8	DIRECTION
First run	int8 is 1.47× slower than fp32	one way
Second run	int8 is 1.75× faster than fp32	the opposite way

Same code, same model, same machine — and the answer completely reversed. Any conclusion drawn from either run alone would have been confidently wrong.

WHY THIS HAPPENS

Look at the timings. In `inproc` mode a call takes between **0.03 and 0.7 milliseconds**. That is about the time it takes to blink.

At that scale, ordinary background activity on your computer — a garbage collector pausing for a moment, the operating system switching tasks, a fan starting — is the *same size as the difference being measured*. You are trying to hear a whisper while someone is rustling paper.

The `http` measurements, by contrast, take 2–25 ms — far bigger than the rustling. Those stayed consistent between runs.

How does it work here?

Confidence intervals — giving every number an honest range

Instead of reporting "1.23× faster", every result is reported as a range: **1.23× [1.15, 1.31]**. This says: the measurement came out at 1.23, and we are reasonably sure the true value lies between 1.15 and 1.31.

Two intervals are built differently, for a reason that matters:

TYPE	USED FOR	WHY IT IS DONE THIS WAY
Bootstrap	Latency percentiles	Latency is lopsided — a few very slow calls drag the average up. A symmetric range would put the lower bound somewhere impossible and understate the upper one. Resampling avoids assuming a shape.
Paired bootstrap	Comparing two things	The two measurements being compared came from the same machine at the same time — same temperature, same background load. Resampling them together preserves that. Comparing two separate intervals is a classic error that makes real differences look like noise.
Wilson interval	Accuracy	The model is 97.41% accurate. A naive interval would give a range like "96% to 101%", and a probability cannot exceed 100%. Wilson stays inside the possible range.

What "bootstrap" means, simply

You have 210 stopwatch readings. You want to know how much your average would wobble if you ran the experiment again.

You cannot run it again a thousand times. But you can do something clever: take your 210 readings and pick 210 at random from them, allowing repeats. Average those. Do that 2,000 times.

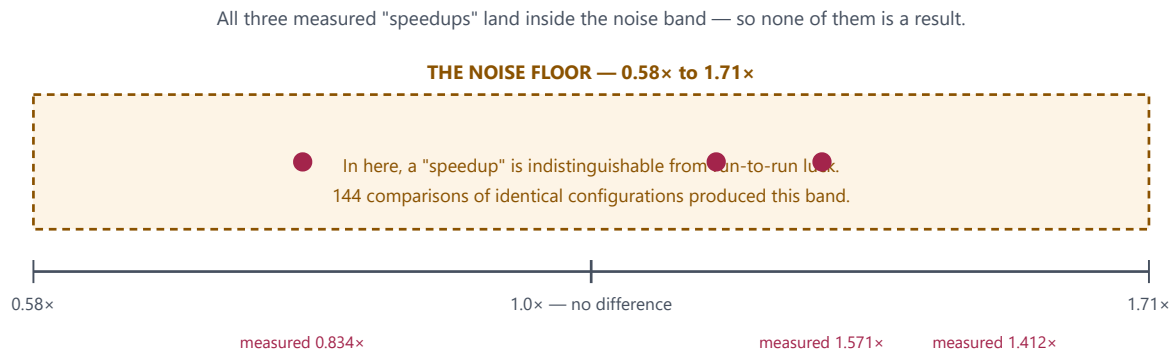
You now have 2,000 slightly different averages. Where most of them land is your confidence interval. You built a picture of your own uncertainty out of the data you already had — no extra runs needed.

The best idea in this project: measuring your own noise

Here is the clever bit that turns a confusing situation into an honest one.

The benchmark already runs every configuration **three times**. So for any one configuration — same runtime, same batch size, same everything — we have three independent measurements of *the identical thing*.

Any difference between those three repeats is, by definition, **the harness measuring nothing at all**. That is the noise floor. It is the answer to "how much difference can this setup produce out of thin air?"



Rule: a comparison only counts if its whole confidence interval sits outside the band.

144 null comparisons made	0.580 lower bound (all modes)	1.708 upper bound (all modes)	19 / 48 comparisons flagged unresolvable
-------------------------------------	---	---	---

The honest headline

THE RESULT THIS PROJECT IS ACTUALLY PROUDEST OF

The harness cannot resolve its own differences. The noise band is **0.58× to 1.71×** — a range of nearly 3× — and every measured "speedup" sits inside it.

So the report flags **19 of 48 comparisons as "unresolvable"** and says so directly, in bold, in the table.

Why this is a success and not a failure: because the alternative is a report that says "int8 is 1.57× faster!" and is wrong. Finding the limits of your own instrument is real engineering work. Pretending not to have limits is how bad decisions get made — and in this case the bad decision would be to switch production to int8 for a speed benefit that may not exist.

Knowing which percentiles are even measurable

There is a second, quieter honesty check. Percentiles need enough data to mean anything.

To report a p99 — the slowest one request in a hundred — you need at least 100 observations before a single one lands in that range. With only 3 observations in the tail, your "p99" is really just "the slowest one we happened to see", wearing a percentile's name.

PERCENTILE	OBSERVATIONS NEEDED	MEASURABLE IN A QUICK RUN?
p50 (the middle)	20	Yes, easily
p90	100	Yes
p95	200	Sometimes
p99	1,000	No — not in a quick run

So the report does not print a p99 and let you assume it is solid. It marks it as underpowered and tells you *how far short* you are — 20 calls short is very different from 20,000 calls short, and you deserve to know which.

WHY WE NEEDED ALL OF THIS

Because a benchmark's job is not to produce numbers. It is to produce numbers a stranger can **act on safely**. A number without its uncertainty is not actionable — it is a rumour with a decimal point.

And the moment you can measure your own error, you gain something unexpected: the ability to say "*I don't know*" with authority. That is a far more valuable professional skill than the ability to produce an impressive figure.

WHERE WILL YOU USE THIS AGAIN?

- **Every performance claim you ever read.** Ask: how many runs? what was the variance? is there a confidence interval? If not, it is marketing, not measurement.
- **A/B testing, load testing, database tuning, front-end optimisation** — all the same problem, all the same tools. Latency is never normally distributed; bootstrapping is the standard modern answer.
- **The noise-floor idea** generalises to any measurement instrument: run it on two identical things and see what it reports. A weighing scale that reads 100 g and 103 g for the same object cannot resolve a 2 g difference, no matter how precise it looks.
- **Wilson intervals for proportions** are used for conversion rates, error rates, click-through rates — anywhere you are measuring a percentage near 0 or 100%.
- **The habit of reporting uncertainty** will make you noticeably more credible than people who do not. It is rare, and it is obvious when you see it.

The smoke alarm: drift detection

A machine-learning service can fail while every technical check says it is healthy. This is the part that catches that, and it is the most machine-learning-flavoured piece of engineering here.

What is it?

A background job that regularly looks at the pictures people have actually been sending, compares them to the pictures the model learned from, and raises a flag if the two have drifted apart.

Why did we need it?

THE PROBLEM BEFORE IT — THE SILENT FAILURE

Imagine your digit model works perfectly. Now imagine that, over months, your users start sending photos taken with a different camera: sharper edges, different lighting, a slightly different way of writing. The model has never seen this style.

What happens? **Almost nothing visible.** The service stays up. Response times are fine. No errors are logged. Every dashboard is green. The model confidently returns answers — it always returns *an* answer, because that is all it can do.

It just gets more of them wrong. Silently. And the only way you find out is when a human eventually notices that the results have been bad for weeks.

Imagine it like this

You learned to recognise your friends by their faces. Now everyone starts wearing sunglasses and hats. You still smile and say "hello, Sarah!" — you are not confused, you are not making an error message. You are just wrong more often, and nobody notices, because you sound perfectly confident.

Drift detection is noticing that everyone is wearing hats now.

How does it work here?

The idea: compare shapes of distributions

You cannot compare individual pictures — tomorrow's pictures are naturally different from yesterday's, and that is fine. What you compare is the overall *shape* of the data.

Imagine it like this

You cannot tell if a class of children has changed by looking at one child. But if you line them all up by height, you get a shape — most in the middle, few at the edges.

Do that every year. If the shape stays the same, the class is the same kind of class. If suddenly everyone is much taller, something changed — even though every individual child is still just "a child."

For each of the 64 pixels, the job asks: *do production values fall into the same buckets, in the same proportions, as during training?*

The measuring stick: PSI

The score is called **Population Stability Index**. It compares the share of values in each bucket now versus during training:

PSI SCORE	VERDICT	MEANING
under 0.1	stable	Looks like training data. Nothing to do.
0.1 to 0.25	watch	Something is shifting. Keep an eye on it.
over 0.25	alert	Meaningfully different. Investigate.

These thresholds are the industry convention, which matters more than it might seem: a PSI of 0.3 means the same thing on a big dataset as on a small one, so the number is portable between teams and companies.

Two detectors, and why only one of them decides

The job computes two different statistics. Only one is allowed to raise the alarm. This is a deliberate design decision worth understanding.

MEASURE	ROLE	WHY
PSI	Decides	It is a <i>magnitude</i> — a size, not a yes/no answer. So it does not care how much data you have. Bigger window, smaller window, same meaning.
KS test	Corroborates	It is a <i>statistical test</i> , which answers "is there <i>any</i> difference?" More data makes it more sensitive — so it eventually finds differences far too small to matter.

THE FALSE ALARM PROBLEM — WHY KS CANNOT BE IN CHARGE

There are 64 features. A statistical test at the usual 5% confidence level falsely fires about once in twenty times *by pure chance*. Across 64 features, that is roughly **3 false alarms every single run** — every run, forever, on data that has not changed at all.

Add the sample-size sensitivity and you get a detector that cries wolf constantly. And the predictable human response to an alarm that fires three times a run is to **mute it**. At which point you have paid for a smoke alarm and disconnected the battery.

So PSI decides. KS is recorded in the output as supporting evidence — genuinely useful when you are investigating a real alert — but it cannot page anyone.

Three features excluded, on purpose

Every 8×8 digit picture has pixels in the corners that are background in *every single training image*. Those pixels never change. Across 64 features, **3 of them are completely constant**.

Those 3 are excluded from scoring, and the reasoning is worth following because it is a nice piece of clear thinking:

- A feature that never varies has no distribution. There is no "normal" for it to deviate from.
- Its PSI is therefore **0 no matter what arrives** — you could send it anything and it would score zero.
- Including it would drag the average PSI down, making real drift in the other 61 features look smaller.
- It would also inflate the denominator in "what fraction of features drifted?" — making a genuine widespread problem look less widespread.

So they are excluded from scoring — but they are *recorded in the output*, so nobody later wonders why the numbers do not add up to 64. Hiding it would be worse than the tiny inconsistency it avoids.

Distinguishing "nothing moved" from "nothing was measured"

A quiet design decision with real consequences. The job publishes a metric called `mlserve_drift_measured`:

VALUE	MEANING
1	The window was compared against the reference and nothing moved. Genuinely healthy.
0	The window was too small to compare. Nothing was checked at all.

Without this distinction those two situations look identical — both show "no drift". So a drift job that had been *broken and silent for three weeks* would look exactly like a perfectly healthy one on every dashboard.

With the distinction, there is a separate alarm: **"the drift detector has not run successfully in two hours."** That catches the failure that a dashboard fundamentally cannot.

WHY WE NEEDED THIS, GENERALLY

"Everything looks fine" and "I have no information" are completely different states, and monitoring systems confuse them constantly. A graph with no line on it is not a flat line at zero. Any time you build a system that reports on itself, ask: *what does this look like when it is broken?* If the answer is "the same as when it is healthy," you have a blind spot.

The alert policy

CONDITION	VERDICT
Any single feature over PSI 0.25	alert
More than 20% of features over PSI 0.10	alert — many features moving together is a real change
Some features over 0.10, none over 0.25	watch — noted, not paged

CONDITION	VERDICT
Nothing over 0.10	stable

The reasoning behind the second rule: one feature drifting is usually noise, a measurement quirk, or a feature that is simply unstable. But when a fifth of your inputs move *together*, that is almost never chance. It means the input has genuinely changed — a new data source, a new camera, a new user population.

WHERE WILL YOU USE THIS AGAIN?

- **Any live model** in any industry needs this: fraud detection, credit scoring, recommendations, demand forecasting, medical risk. The features differ; the technique is identical.
- **PSI and KS** are standard tools in data science and analytics roles. Recognising which one answers which question is a genuine interview differentiator.
- **The multiple-comparisons problem** (64 tests means 3 false alarms) appears everywhere data is tested in bulk: A/B testing many metrics, genomics, anomaly detection across many sensors.
- **Distinguishing "no problem" from "no data"** is a principle for every dashboard, alert and health check you will ever write.
- **The "false alarms get muted" argument** is how you justify spending engineering time on alert quality. An alert nobody trusts is worse than no alert, because it trains people to ignore the system.

The photocopy you actually test: backup

Everyone makes backups. Almost nobody knows whether theirs work. This part is about the difference — and it is one of the most practically valuable pieces in the whole project.

What is it?

Two programs. One makes a dated archive of everything important. The other takes that archive, *actually restores it*, and checks that what came back matches what went in.

Why did we need it?

THE PROBLEM BEFORE IT

Writing a backup is easy. A single command, or ten lines of code. It produces a file. Everyone feels safe.

But a backup file is a **hypothesis**, not a guarantee. It has never been proven to contain what you think. The proof only arrives on the worst day of your professional life: when production data is gone and you reach for the archive.

And that is exactly the moment when you find out the file was truncated, or the database inside it was captured mid-write and is corrupt, or the restore procedure needs a tool you do not have, or the archive was being written to a directory that stopped existing six months ago.

A backup you have never restored is not a backup. It is a wish.

How does it work here?

What actually gets proven

The verifier does not check that the file exists. That would be nearly worthless. It does all of this:

#	CHECK	WHAT IT WOULD CATCH
1	Open the archive as a real archive	A file that was truncated, or is not actually an archive at all.
2	Find the manifest inside	An archive from a different version, or an empty one.
3	Verify the checksum of every model file	Silent corruption — a single flipped bit in the model file, which would make it load and quietly predict rubbish.
4	Verify the database file checksum	The same, for the data.

#	CHECK	WHAT IT WOULD CATCH
5	Actually restore the database into a scratch location	A file that is structurally valid but unusable in practice.
6	Count the rows in the restored copy and compare to the manifest	A restore that "succeeded" but lost data.

Only after all six does it report success — and it **raises an error on any mismatch** rather than returning a true/false that a caller could ignore.

The bug this design would have caught

The database is copied using SQLite's own backup facility, not by copying the file. That sounds pedantic. It is not.

WHY COPYING A DATABASE FILE IS NOT A BACKUP

A live database is not just the file on disk. Part of its current state lives in memory — pages waiting to be written, a transaction in progress. If you copy the file while it is being written to, you can capture a **torn** state: half of the old data and half of the new.

The result looks like a perfectly normal database file. It opens. It answers queries. It is just quietly missing or duplicating rows at the moment of the copy.

SQLite's backup API takes a proper, consistent snapshot — it knows how to do this correctly. Using it is one line instead of one line, so there was never a reason to do it the wrong way.

Testing the alarm, not just the safe

There is a subtle testing principle here. A verifier that has only ever been seen to *pass* has not been tested. It might pass everything — including a corrupt archive.

So the test suite deliberately corrupts an archive and asserts that the verifier **rejects** it. That is the only way to know the verifier actually checks anything.

Imagine it like this

You install a smoke alarm. You press the test button. It beeps. Good — the alarm works.

But how do you know it would beep for actual smoke, and not only when you press the button? You have to test it with something that is genuinely a problem. That is what corrupting an archive does for this verifier.

Why it is written in Python, not shell

The original plan was a shell script, which is the traditional way to do this. It was changed to Python for one reason: **the developer's machine is Windows, where shell scripts cannot be tested.**

A backup routine that has never been run is a liability — and the backup path is the *last* place you want untested code. Writing it in Python means it can be run and tested on every machine, in the test suite, and in the automated checks. The Ansible setup runs this same code on a timer, so nothing is lost.

WHERE WILL YOU USE THIS AGAIN?

- **"Verify your restores"** is one of the most respected things you can say in a systems interview. It is the standard answer to "how do you know your backups work?" and most candidates do not have it.
- **Checksums for integrity** are used everywhere: package managers, download verification, container images, data pipelines, git itself.
- **Testing the failure path** — deliberately breaking something to prove the detector notices — is a core discipline. The same idea appears in chaos engineering at large companies.
- **Choosing a testable language over a traditional one** happens whenever a script matters enough that being untested is unacceptable. Automation, migration scripts, deployment steps.
- **Row-count and checksum comparison** is how you verify any data migration: database upgrades, cloud moves, format changes.

The settings drawer, and the remote control

Two small pieces of housekeeping that make everything else pleasant to use. Neither is exciting; both are the difference between a project people can work with and one they avoid.

What is it? — the settings drawer

Four small text files in `configs/` that hold every number the project uses. Want to try 128 pictures per batch instead of 64? Change one line in a text file. No code editing, no searching through thousands of lines for a magic number.

FILE	HOLDS
<code>configs/model.yaml</code>	How big the network is, the seed (1337), how the data is split.
<code>configs/serving.yaml</code>	Which runtime is the default, the maximum batch size, whether to log requests.
<code>configs/bench.yaml</code>	The whole benchmark matrix: how many calls, how many repeats, batch sizes, concurrency levels.
<code>configs/drift.yaml</code>	Bucket count, minimum window size, and the PSI thresholds.

Why did we need it?

THE PROBLEM BEFORE IT

Numbers scattered through the code are one of the most common sources of quiet confusion. Someone changes a batch size in one place and not another. Two parts of the system disagree about how big a batch is allowed to be, and the bug is invisible until something fails in production.

Worse: nobody can see the whole picture. What *are* the current settings? You would have to read the entire codebase to find out.

With config files, everything important is visible in four short files. And each one carries its reasoning as comments — why the batch limit is 64, why the seed is 1337, why the bucket count is 10. Six months later, those comments are what stops someone from "fixing" a number that was deliberate.

A detail that matters: environment variables

Some settings can be overridden without editing the file at all, using *environment variables* — small named values that a program reads when it starts.

```
# Run a quick benchmark in CI without touching the config
MLSERVE_BENCH_CALLS=10 MLSERVE_BENCH_REPEATS=1 python tasks.py bench

# Write results somewhere other than ./results
MLSERVE_RESULTS=/tmp/myrun python tasks.py bench
```

This is how one configuration serves two purposes: a real, thorough run on a laptop, and a fast smoke test in automation. The file holds the settings a report is pinned to; the environment handles temporary variation.

A REAL BUG THIS CAUSED, AND ITS LESSON

There is a setting that decides where results get written. On a developer's machine it defaults to a folder inside the project, which is writable. Inside the container, that same default pointed at a folder **owned by the system administrator account**, and the service deliberately runs as an unprivileged user.

Result: the drift job crashed with "permission denied", every time, only in the container. It worked perfectly on every developer machine — which is why it survived so long.

The lesson: "it works on my machine" is usually not carelessness. It is a genuinely different environment. Defaults that resolve relative to the code's location behave differently when the code is moved somewhere with different permissions. This is why the fix was to set the path *explicitly* in the container, with a comment explaining why the line must not be deleted.

What is it? — the remote control

A single file, `tasks.py`, that gives every project action a short name.

```
python tasks.py test      # run the checks
python tasks.py bench    # run the benchmark
python tasks.py report    # build the report
python tasks.py verify    # do everything, end to end
```

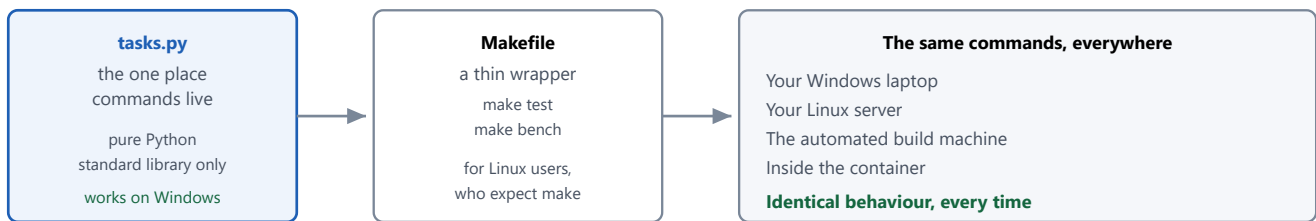
Why did we need it?

THE PROBLEM BEFORE IT

Real projects need long, fiddly commands with the right environment, the right paths and the right order. Nobody remembers them. People copy them out of old chat messages and get them slightly wrong.

And there is a worse problem: the commands live in one person's memory. A new contributor has to be told everything.

With a task runner, there is one place to look. Running `python tasks.py` with no arguments **lists every available action with a description**, so the project explains itself.



One definition of each command, three ways to reach it.

WHY PYTHON INSTEAD OF THE USUAL TOOL

Everyone in this field has heard of `make`, which is the traditional way to do this. But `make` is not installed on Windows by default, and this project must work on the developer's actual machine.

Installing extra software to run your own project is friction, and friction is what makes people stop working on things. So the tasks are written in plain Python using only the standard library — which means if you have Python, you can run the project. No installation step at all.

A Makefile is provided too, so people on Linux who instinctively type `make test` are not surprised. It simply forwards to the same Python file. **Two doors, one room.**

WHERE WILL YOU USE THIS AGAIN?

- **Task runners of some kind exist in every mature project** — `make`, `just`, `npm scripts`, `rake`, `invoke`, `Taskfile`. The concept is universal even when the tool differs.
- **Config files with comments explaining the reasoning** are how you leave notes for the future. Code tells you *what*; a comment next to a threshold tells you *why it is that number and not another*.
- **Environment variable overrides** are the standard way to configure the same software for development, testing and production without maintaining separate copies.
- **Choosing your own tools to fit the environment** rather than following tradition — and explaining the choice — is a judgement you will be asked to make repeatedly.

The lunchbox: Docker

The tool that ended the phrase "but it works on my machine." This part explains what containers actually are, and the specific decisions that make this particular image good.

What is it?

A way of packaging a program together with *everything it needs to run* — the right version of Python, the right libraries, the right system settings — into a single sealed unit.

Imagine it like this

Your friend says "I made you lunch, come get it." You arrive and they say "oh, but you need to bring your own plate, and a fork, and my kitchen only works on Tuesdays."

A lunchbox solves this. The food, the plate, the fork and the napkin are all inside. Hand it to anyone, anywhere, and they can eat. That is what Docker does for software.

Why did we need it?

THE PROBLEM BEFORE IT

Our program needs Python, plus about eight libraries, each with its own specific version. Different versions of those libraries behave differently. Your laptop has one combination; the server has another; the automation machine has a third.

So the program works on your machine and mysteriously fails elsewhere. Hours disappear into "which version of numpy do you have?"

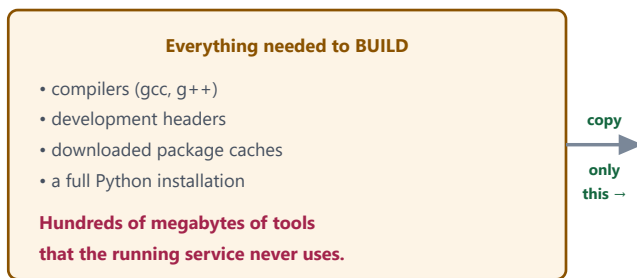
Containers make the environment part of the software. There is no "which version do you have" — the version is fixed inside the box.

How does it work here?

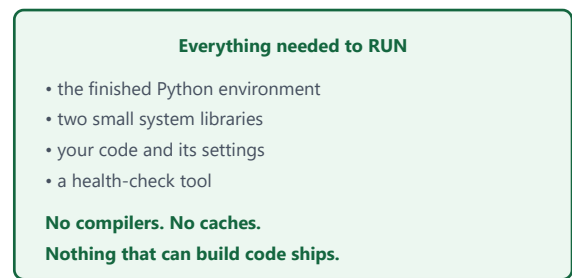
The two-stage build

This is the most interesting decision in the Dockerfile, and it solves a problem that surprises people.

STAGE 1 — THE WORKSHOP (thrown away afterwards)



STAGE 2 — THE LUNCHBOX (what actually ships)



Build in one box, ship a different one. The compilers never reach production.

WHY THIS MATTERS FOR SECURITY, NOT JUST SIZE

Compilers are tools for turning source code into programs. If an attacker gets into your container, a compiler is one of the most useful things they could find — it lets them build their own tools inside your system.

Removing compilers from the shipped image is therefore both smaller *and* safer. This is a standard practice with a name — minimising the attack surface — and it is exactly what the two-stage build achieves almost for free.

Four more decisions, each solving a real problem

DECISION	PROBLEM IT SOLVES
Install torch from the CPU-only source	The default version of the machine-learning library ships with GPU support and pulls in roughly 2.5 GB of graphics-driver libraries. This service runs on a processor and never touches a GPU. Pointing the installer at the CPU-only source is one line, and removes gigabytes of dead weight. <i>Verified: zero GPU packages in the final image.</i>
Run as a non-root user	By default, container programs run with full administrator rights. If someone finds a way in, they own everything. Running as an ordinary restricted user means a break-in is contained. Costs one line; is a genuine security boundary.
Create the data folders as the service user	Containers can only write to folders they are given, and a folder that does not already exist gets created by the administrator account. The service then cannot write to its own data directory. Creating the folders up front, owned by the right user, prevents a whole category of confusing permission failures.
Check writability at build time	One extra line that tries to write a file into each data folder during the build. If any folder is not writable, the build fails immediately — rather than the service failing at 2am. Failing fast, early, and loudly is almost always the right trade.

The health check that was deliberately removed

The Dockerfile contains a comment explaining something that is *not* there:

```
# No HEALTHCHECK here, deliberately. This image is used by five different roles
# (init, api, drift, drift-publisher, backup) and only one of them serves HTTP,
# so an image-level healthcheck is wrong by construction.
```

WHAT WENT WRONG

The image originally had a health check that tried to reach a web address. Five different jobs use this image, but only *one* of them is a web service. The other four — the model builder, the drift job, the backup job — have no web address to check.

So four out of nine containers permanently reported "**unhealthy**" — while working perfectly. Worse, the command that starts everything up and waits for health checks *never returned*. Starting the project appeared to hang forever.

The fix: move health checks out of the image and into the file that describes the stack, where each service can be checked for what it actually does. A liveness check belongs to the thing being checked, not to the tool it was built with.

WHERE WILL YOU USE THIS AGAIN?

- **Docker is close to universal** in backend, DevOps, platform and data roles. Understanding images versus containers, layers, and multi-stage builds is expected knowledge.
- **Multi-stage builds** are the standard way to keep production images small in any compiled language — Go, Rust, Java, C++.
- **Running as non-root** is required by many security policies and is a frequent security-review finding when missed.
- **Failing the build rather than the runtime** is a general principle: catch problems as early as possible, when they are cheap and obvious.
- **Putting the check where the knowledge is** — the health check next to the service definition, not in the shared image — is a design instinct that applies far beyond Docker.

The whole kitchen: Docker Compose

Nine separate programs that must start in the correct order, find each other, and share data. This is the file that describes the entire system in one place.

What is it?

A single text file listing every piece of the running system — what each one is, how it is configured, what it needs, and in what order things must happen.

Why did we need it?

THE PROBLEM BEFORE IT

Our system is not one program. It is nine: the model builder, the API, the drift job, the backup job, four monitoring programs, and a small test server. Each one needs the right settings, the right folders connected, and the right ports.

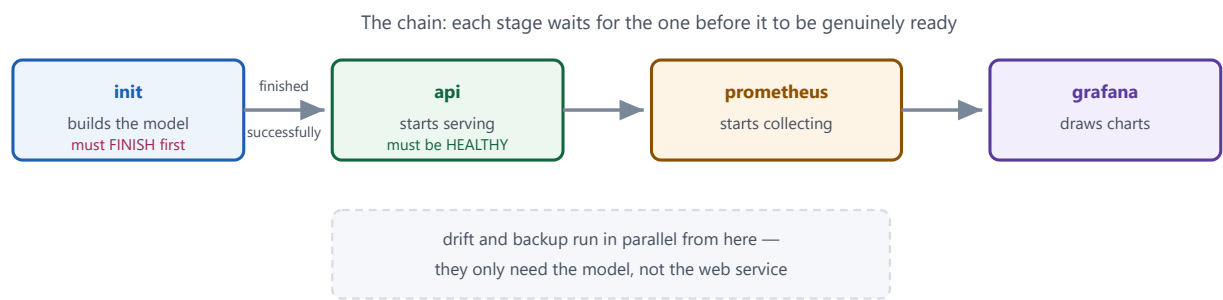
Without a single description, starting the system means typing nine long commands in the right order, remembering every setting, and hoping you got them all right. Nobody can reproduce that, and nobody can review it.

How does it work here?

The nine containers

CONTAINER	JOB	WHAT IT DOES
init	one-off	Trains the model, exports the three runtimes, builds the drift reference. Then stops.
api	permanent	The shop counter. Answers prediction requests.
drift	loop	Checks for drift every 5 minutes.
backup	loop	Backs up and verifies every hour.
prometheus	permanent	Collects and stores all the numbers.
alertmanager	permanent	Decides who to tell when something is wrong.
grafana	permanent	Draws the dashboards humans look at.
node-exporter	permanent	Reads the drift job's numbers from a file and republishes them.
alert-sink	permanent	A tiny test server that receives alerts, so we can prove alerting works end to end.

The startup order — and why it cannot be guessed



Two different kinds of "wait": wait for another job to finish, versus wait for another service to be healthy. Conflating them causes flapping.

THE RACE CONDITION THIS PREVENTS

The API cannot serve predictions without model files. If both containers started at the same time, there would be a window where the API is running but its model files do not exist yet. It would report "not ready" — correctly — and then become ready a few seconds later.

That sounds harmless. It is not, because it happens on *every single start*. Automated systems that check health see a service that is intermittently unavailable, and may make decisions based on that — restart it, take it out of rotation, page someone.

Waiting for the model builder to **finish successfully** removes the window entirely. The API never exists in a state where it cannot serve.

Data that must outlive the containers

Containers are disposable. Delete one and everything inside it is gone. But the model files, the request log, the results and the backups must survive. So each gets a **named volume** — a storage area that exists separately from any container.

VOLUME	HOLDS	WHO CAN WRITE
artifacts	The three model files, the drift reference	init writes; api and drift read only
data	The request log database	api, drift
results	Benchmark output, drift report	drift, init
backups	Backup archives	backup
drift-textfile	The drift numbers, in a format Prometheus can read	drift writes; node-exporter read only

Note the read-only ones. The API does not need to *change* the model files — only read them. So it is given read-only access. If the API were ever compromised, it could not corrupt the model the whole system depends on. That is one word in a config file buying a real safety property.

Small decisions with outsized effects

Ports are adjustable

Everything the project listens on can be moved with a setting. Most developers already have something on port 8000 or 3000. A project whose first instruction fails with "port already in use" makes a bad first impression.

Timers are loops, not cron

The drift and backup jobs are written as a simple loop with a sleep. One less program to run, and the interval is visible on a single line instead of buried in a scheduling file.

An exit code of 2 is not a failure

The drift job exits with code 2 when it finds drift. That is a *finding*, not a crash. The loop distinguishes them, so a real drift alert does not look like a broken container.

Postgres is commented out on purpose

A second database is included but disabled, with instructions. The restore-verification code only supports the primary one — and shipping a database whose backup cannot be proven would be worse than not offering it.

WHERE WILL YOU USE THIS AGAIN?

- **Compose for local development is nearly universal.** Almost every backend project has one. Being able to read and write one is a baseline skill.
- **Startup ordering and health gates** carry directly into Kubernetes (readiness probes, init containers) — the concepts are identical, only the vocabulary changes.
- **Named volumes and read-only mounts** are how state is managed in all container systems.
- **Not shipping what you cannot verify** — the disabled Postgres — is a judgement call interviewers notice. It is more impressive than offering every option.

The dashboard and the alarm bell: monitoring

Three programs that answer three different questions: what is happening, what does it look like, and who needs to know.

What is it?

Prometheus — the record keeper

Every 15 seconds it asks the API "what are your numbers?" and stores the answers with timestamps. It keeps 15 days of history.

Alertmanager — the decision maker

When a rule fires, this decides who to tell, whether to repeat, and when to stop. It handles the annoying human problems: not sending 400 copies of the same alert.

Grafana — the picture maker

Turns stored numbers into graphs a human can read at a glance. Pre-configured, so it works the moment you open it.

alert-sink — the tester

A deliberately tiny web server whose only job is to receive alerts and print them. It exists so we can prove the whole alerting chain works, rather than assuming it does.

Why did we need it?

THE PROBLEM BEFORE IT

Without monitoring, you learn about problems from users. "The app is slow" is what you get instead of "response times crossed 50 ms at 14:02."

And without alerting, nobody knows at 3am. The service could be fully broken and the first anyone hears is at 9am when someone tries to use it.

How does it work here?

The numbers the API publishes

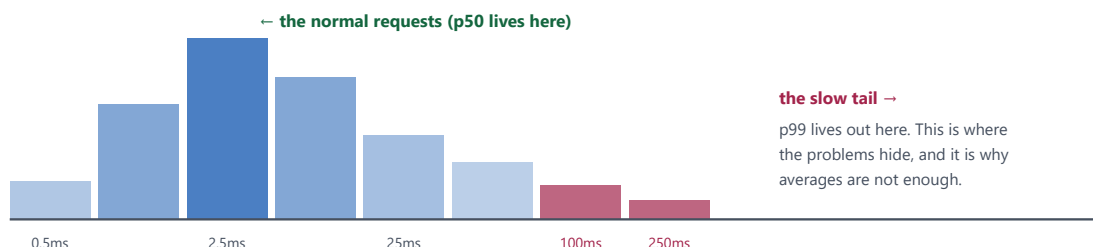
METRIC	ANSWERS
<code>mlserve_requests_total</code>	How many requests, broken down by endpoint and result.
<code>mlserve_request_duration_seconds</code>	How long they took — this is what produces p50, p95 and p99.
<code>mlserve_batch_size</code>	How many pictures arrived per request.
<code>mlserve_model_loaded</code>	Is the model actually loaded right now? 1 or 0.

METRIC	ANSWERS
<code>mlserve_model_size_bytes</code>	How big each runtime's file is.
<code>mlserve_validation_errors_total</code>	How many requests were rejected as malformed.
<code>mlserve_inference_errors_total</code>	How many times the model itself failed.
<code>mlserve_db_write_errors_total</code>	How many times a log write failed — a signal the database is in trouble.
<code>mlserve_predictions_total</code>	What the model has been predicting, by digit. If it starts saying "3" for everything, this shows it.

Special values called "histograms"

Latency is not a single number — it is a distribution. A histogram stores how many requests fell into each speed range. That is what allows the system to calculate "how slow was the slowest one in a hundred?"

HOW MANY REQUESTS TOOK HOW LONG



A latency histogram. The interesting part is not the peak — it is how far to the right the tail stretches.

The alert rules

RULE NAME	FIRES WHEN
<code>MLServeP99LatencyBreach</code>	p99 latency above 50 ms for 5 minutes.
<code>MLServeP99LatencyCritical</code>	p99 above 250 ms. Something is badly wrong.
<code>MLServeModelNotLoaded</code>	The model is not loaded. The service cannot do its job at all.
<code>MLServeErrorRateHigh</code>	Too many requests failing.
<code>MLServeInferenceErrors</code>	Any inference error at all in 10 minutes. Even one is worth knowing about.
<code>MLServeDbWriteFailures</code>	Log writes failing — the database is in trouble, and drift detection loses its data.
<code>MLServeDriftDetected</code>	The drift job found real drift.
<code>MLServeDriftDetectorStale</code>	The drift job has not run in 2 hours. The detector is broken.
<code>MLServeValidationErrorSpike</code>	Sudden rush of malformed requests — usually a broken client, sometimes an attack.

RULE NAME	FIRES WHEN
<code>MLServeTailPowerLow</code>	Not enough data to compute a meaningful p99. <i>An honesty check in the monitoring itself.</i>

That last one deserves a moment. Most monitoring systems will happily draw a p99 from four data points and present it as fact. This one raises a flag saying "there is not enough data for this number to mean anything" — the same honesty principle from Part 7, carried all the way into the live dashboard.

THE AWKWARD PROBLEM: THE DRIFT JOB HAS NO WEB ADDRESS

Prometheus collects numbers by asking services over the network. But the drift job is not a service — it runs, does its work, and exits. There is nothing to ask.

The traditional answer is a "push gateway": the job pushes its numbers somewhere and Prometheus reads them from there. That means running *another* container, plus the constant risk that Prometheus reads the numbers at a bad moment.

Our solution is simpler: the drift job writes its numbers to a file, and a small existing component reads that file. Two lines of configuration, no extra container, no timing window.

And the file is written by creating a temporary file and then *renaming* it — because renaming is instant and atomic. A reader either sees the old complete file or the new complete file, never a half-written one. Without this, Prometheus would occasionally read a partial file, fail to parse it, and produce a random dip in your graphs.

WHERE WILL YOU USE THIS AGAIN?

- **Prometheus and Grafana** are close to industry standard for infrastructure monitoring. Reading a PromQL query or editing a dashboard is a common job requirement.
- **The three-way split** — collect, decide, display — is the right mental model for any monitoring system, including cloud-native ones where the names differ.
- **Alert rules need as much thought as code.** Every rule you write is a promise to wake someone up. Rules that fire when nothing is actionable actively make systems less reliable.
- **Monitoring your monitoring** — the "detector is stale" rule — is the mark of someone who has been on call. Systems fail in ways their own dashboards cannot show.
- **Atomic file writes via rename** is a genuinely useful trick for any situation where one program writes and another reads: log files, caches, lock files, config reloads.

The robot installer: Ansible

How you set up one server — or a hundred — without doing any of it by hand, and without forgetting step 7 next time.

What is it?

A tool that takes a written description of how a computer should be set up, and makes the computer match it. It connects over the network, checks what is already there, and only changes what is wrong.

Imagine it like this

You are moving into a new house. Instead of walking around deciding where things go, you write a checklist: "the kettle goes on the left counter, the plates in the top cupboard, the towels in the bathroom."

*Now you hand that list to a helper. They set up the house. And the magic part: if you run the list again tomorrow, they check each item, see it is already correct, and **change nothing**. That is the whole idea.*

Why did we need it?

THE PROBLEM BEFORE IT

Before the project can run, a server needs: the right user accounts, a firewall configured, the container engine installed, the code fetched, services installed with timers, log rotation set up. That is a long list of steps.

Done by hand, three problems appear immediately:

- **It is not repeatable.** The second server will be set up slightly differently, because you will forget something or do steps in a different order.
- **It is not reviewable.** Nobody can check your work. There is no record of what you did.
- **It is not recoverable.** If a server dies, rebuilding it means doing everything again from memory.

How does it work here?

Six roles, each doing one job

ROLE	WHAT IT SETS UP
base	Basic system packages and settings that everything else assumes.
users	Service accounts and their permissions.
firewall	Which network ports are open and which are closed. Deny by default.

ROLE	WHAT IT SETS UP
docker	The container engine itself.
deploy	The application: fetch the code, install the timers and services that run it.
logrotate	Stops log files growing until the disk fills.

The word *role* just means "a reusable bundle of instructions that belongs together." It means you can apply the firewall rules to any server, not only this one — the same bundle works for a different project entirely.

The key idea: describe the destination, not the journey

THE OLD WAY — commands

```
install nginx
mkdir /etc/myapp
cp config.yml /etc/myapp/
systemctl start nginx
```

Run it twice?

It fails, or does damage — the folder already exists, the service is already running. You must write "check whether it is already done" around every step.

THE ANSIBLE WAY — a description

```
nginx is installed
/etc/myapp exists
config.yml is at /etc/myapp/
the nginx service is running
```

Run it twice?

Nothing happens the second time. It checks, sees everything already matches, and reports "**changed: 0**".
That is the proof it worked.

Commands describe a journey. A good configuration describes a destination — and can be run any number of times safely.

WHY "CHANGED: 0" IS THE GOAL

When you run the setup a second time on an already-correct machine, the ideal output is **changed: 0, failed: 0** — meaning "everything was already exactly right, I did nothing."

This is not a technicality. It is a real test of whether the description is complete and correct. If running it twice changes things, the description does not fully determine the outcome — which means two servers set up from it could end up different. **Idempotent** is the formal word, and it is the property that makes automation trustworthy.

What we proved, and what we could not

CHECK	RESULT
The configuration is syntactically valid	Proven — passes
All 56 tasks in 6 roles resolve correctly	Proven — listed and verified
Best-practice linter finds no problems	Proven — clean
Actually setting up a real, live machine	Not proven here

The last row is the honest gap. Testing the real thing requires a virtual machine, which on this Windows machine requires a system component that is not currently enabled. It is not a flaw in the configuration — it is simply not exercised.

So the project says exactly that: *"the configuration is verified to be well-formed and complete; it has not been applied to a real machine."* It would be easy to phrase that as "fully verified Ansible deployment" and most readers would not check. Stating the limit is the entire point of the project.

A real problem we hit — and its lesson

TWO VERSIONS OF THE LINTER DISAGREED

The automated build machine used one version of the quality-checking tool; the local machine had an older one. They gave different answers about the same files.

This is a genuinely nasty class of problem: **the check passes for you and fails for everyone else**, with no code difference to explain it. It wastes hours, and it erodes trust in the automation.

The fix has two parts. First, run the newer version locally to make the two agree. Second, understand *why* the newer version complained — in this case, it wanted variable names to be more specific, so that a variable belonging to one role does not accidentally collide with a similarly-named variable in another role. That is a real robustness improvement, not nitpicking.

The lesson: pin your tool versions, and when a check disagrees between two environments, treat that as a genuine finding rather than an annoyance to silence.

WHERE WILL YOU USE THIS AGAIN?

- **Ansible is one of the three big configuration tools** (with Puppet and Chef) and is the most common in cloud and DevOps roles. Knowing roles, idempotency and inventory is directly marketable.
- **Idempotency as a concept** applies far beyond servers: database migrations you can re-run, deployment scripts, API operations, message processing. It is one of the most valuable ideas in distributed systems.
- **Configuration as code** — storing your infrastructure setup in version control, where it can be reviewed and rolled back — is standard practice everywhere.
- **Being explicit about untested paths** will come up every time you ship automation. "I verified the script parses but did not run it against production" is a complete, honest sentence that colleagues can act on.

The robot builder: Terraform

Ansible sets up a computer that already exists. Terraform creates the computer in the first place. Together they cover the whole life of a server.

What is it?

A tool where you write down what computers you want, and it talks to your cloud provider to make them exist — networks, servers, addresses, firewalls and all.

Imagine it like this

Ansible is the helper who furnishes your house. Terraform is the builder who builds the house — and who will also knock it down cleanly when you are done with it.

Why did we need it?

THE PROBLEM BEFORE IT

You need a server. So you log into a cloud website, click through a form, choose a size, pick a region, click create. It works. Now do it again for the next environment.

Two problems. First, nobody wrote down what you clicked — there is no record, no review, no way to make the same thing again exactly. Second, when you are finished experimenting, the server keeps running and **keeps costing money**, because nothing told the cloud to stop.

How does it work here?

The same description, two destinations

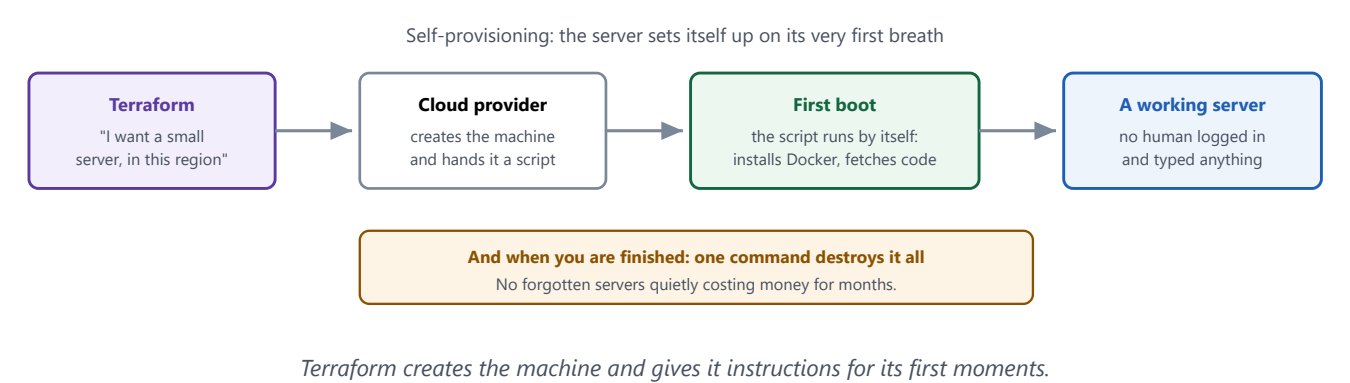
The infrastructure is written once as a reusable module, then used by two different environments:

ENVIRONMENT	CREATES	PURPOSE
local	A virtual machine on your own computer	A place to practise and test without paying anything or touching the internet.
free-tier	A real cloud server on a free allowance	Proves the same description works against a real provider.

This is the point of writing infrastructure as code: **the same description produces the same shaped thing** in both places. Testing locally tells you something about what will happen in the cloud.

The clever part: a server that sets itself up

Creating a server and then configuring it are two separate steps. But there is a neat trick that merges them: when the server is created, you hand it a *script to run the moment it first boots*.



What we proved, and how

CHECK	RESULT
The provider plugin downloads correctly	Proven
The description is valid — for both environments	Proven
Terraform can plan the change without doing it	Proven — "2 to add, 0 to change, 0 to destroy"
Formatting follows the standard style	Proven
The plugin version is locked down	Proven — lock files committed

TWO IDEAS WORTH STEALING

- Plan before applying.** Terraform can tell you exactly what it is *about to* do, without doing it: "2 things will be created, 0 changed, 0 destroyed." That is a safety net no manual process has. You get to read the consequences before you accept them.
- Lock the plugin version.** The tool relies on an add-on written by someone else. If that add-on publishes a new version with a bug, your infrastructure breaks for reasons that have nothing to do with your code. Committing a lock file means you only move versions when you choose to. This is the same lesson as the Ansible version mismatch — *pin your dependencies* — showing up in a different place.

WHERE WILL YOU USE THIS AGAIN?

- **Terraform is the most widely used infrastructure-as-code tool** and appears in nearly every cloud job advertisement. It works with AWS, Azure, Google Cloud and dozens of others.
- **The plan/apply pattern** — preview the change, then confirm it — is the correct way to make any risky change.
- **Self-provisioning machines** are how auto-scaling works. When load rises, new servers appear and must configure themselves with nobody watching.
- **Being able to destroy cleanly** is a genuine engineering concern: orphaned cloud resources are a common and expensive problem in real companies.
- **Lock files** appear in every ecosystem — `package-lock.json`, `poetry.lock`, `Gemfile.lock`. The reason is always the same.

The homework checker: CI/CD

Robots that check your work every time you change it — so that problems are found in minutes, by a machine, instead of in production, by a customer.

What is it?

Continuous Integration means: every time you save your work to the shared repository, a robot automatically checks it. **Continuous Delivery** means: if all the checks pass, the robot can also publish or deploy it.

Imagine it like this

You hand in homework. Instead of waiting a week to find out you got question 4 wrong, a helper checks it the moment you finish, every time, and tells you immediately.

An hour later you have already forgotten what you were thinking when you made the mistake. A week later you have no idea at all. Checking immediately is worth ten times more than checking eventually.

Why did we need it?

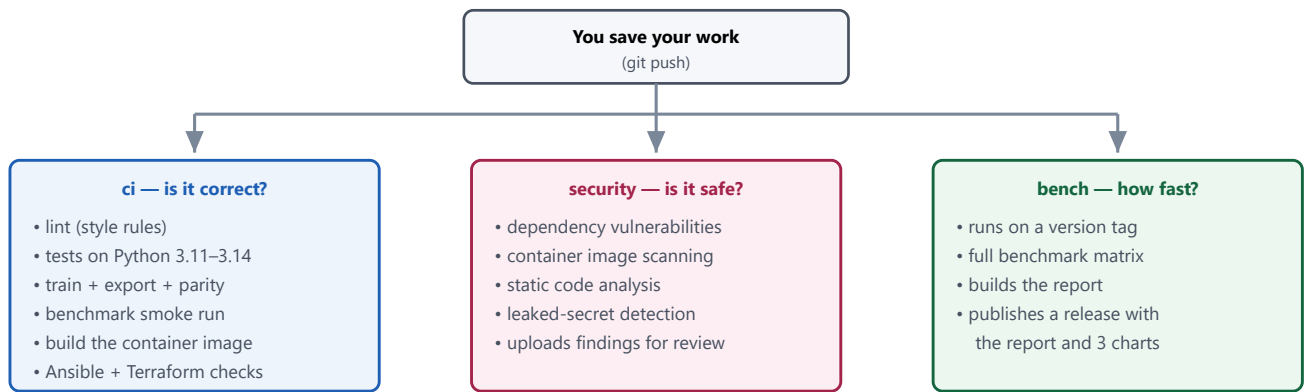
THE PROBLEM BEFORE IT

Your checks only run when you remember to run them. And you will forget — not through carelessness, but because at 1am when you are fixing one last thing, running the full test suite is the step that feels skippable.

Then it becomes someone else's problem. Broken code reaches the shared branch, and everyone who pulls it gets a broken project. The person who finds out is never the person who broke it.

How does it work here?

Three separate robot jobs



Three independent questions, three independent robots. Any one failing blocks the change.

Why the benchmark runs on a tag, not on every change

The full benchmark takes several minutes and produces a report with charts. Running it on every single save would be wasteful and would produce dozens of near-identical reports nobody reads.

So it runs when you mark a **version** — a tag like `v0.1.0`, meaning "this is a real release." The report and the charts get attached to that release permanently, as the record of what that version measured on the day it was published.

WHY THIS PROJECT AVOIDS PAID SERVICES ENTIRELY

The rule for this project is **zero cost**. So there is no paid monitoring service, no paid build runner, no managed database. Everything runs on free allowances or on the developer's own machine.

This is not only about money. It is also a stronger demonstration: an engineer who can build a complete, monitored, tested, deployable system with free tools has shown they understand what each piece is really *for*. Anyone can buy a service. Knowing which service you would buy, and why, requires the understanding this project is trying to build.

The robot that updates dependencies

There is a fourth robot that does something different: it watches for new versions of the tools the project uses, and proposes an update for each one.

ROBOT PROPOSES AN UPDATE TO	WHY
The actions used in the checking itself	Keeps the checking machinery current and secure.
Python libraries	Security fixes and bug fixes in dependencies.
The base container image	Operating-system security patches.

Each proposal arrives as a small, separate change you can review and accept or reject. That is much safer than updating everything at once, because if something breaks you know exactly which change caused it.

WHY NOT ACCEPT THEM ALL AUTOMATICALLY?

Because updates can genuinely break things, and the project's own history proves it. When the base image moved from Python 3.12 to 3.14, the automatic checks *all passed* — but they had been testing Python up to 3.13 only. The version actually being deployed was never tested.

Everything was green. The gap was invisible. This is the most dangerous kind of problem: **a check that passes for the wrong reason.**

The fix was not to stop accepting updates. It was to add a check that reads the container's stated Python version and *refuses to pass unless that version is in the tested list*. Now the two cannot drift apart silently again — in either direction.

WHERE WILL YOU USE THIS AGAIN?

- **CI is universal.** GitHub Actions, GitLab CI, Jenkins, CircleCI, Azure Pipelines — every professional team has one, and every interview will touch it.
- **Splitting checks into "correct", "secure" and "performance"** mirrors how real teams organise their pipelines, and how they think about release readiness.
- **Automated dependency updates** are now standard practice and a genuine security control. Most breaches exploit known vulnerabilities in outdated dependencies.
- **"A check that passes for the wrong reason"** is a phrase worth remembering. When something is green, ask what it would take for it to go red. If you cannot answer, it may be checking nothing.

How we know it works: the tests

255 automatic checks. This part explains what they are for, and why the most valuable ones are the ones that check for failure.

What is it?

Small programs that run your code and check it behaves as expected. Run them all with one command; if anything is wrong, you are told immediately.

Why did we need it?

THE PROBLEM BEFORE IT

Software changes constantly. Every change risks breaking something that used to work — and the connection is often not obvious. You fix a display bug and break the backup job.

Tests are how you make changes *confidently*. Without them every change is a gamble, and the natural response is to become afraid of changing anything. That is how projects die: not from bugs, but from nobody daring to touch them.

How does it work here?

Fourteen test files, each guarding one area

TEST FILE	WHAT IT PROTECTS
<code>test_model.py</code>	The model trains, the data splits correctly, no leakage.
<code>test_api.py</code>	Every endpoint behaves correctly — including for bad input.
<code>test_schema.py</code>	Requests are validated; wrong shapes are rejected.
<code>test_registry.py</code>	All three runtimes load and predict consistently.
<code>test_stats.py</code>	The statistics are mathematically correct.
<code>test_report.py</code>	The report is generated correctly from raw data.
<code>test_drift.py</code>	Drift is detected when present, and <i>not</i> when absent.
<code>test_backup.py</code>	Backups restore correctly — and corruption is caught.
<code>test_monitoring.py</code>	Monitoring configuration is valid and wired together correctly.

TEST FILE	WHAT IT PROTECTS
<code>test_ansible.py</code>	The server setup description is complete and well-formed.
<code>test_terraform.py</code>	The infrastructure description is valid for both environments.
<code>test_packaging.py</code>	The container and build files are consistent with each other.
<code>test_alert_sink.py</code>	Alerts can actually be delivered.
<code>test_seed.py</code>	Test data generation works.

The most valuable kind of test: checking that things fail correctly

Most tests check that things work. The best tests check that things *fail properly* — because failure paths are rarely exercised by hand and are where the surprising bugs live.

TEST	WHAT IT PROVES
Send a deliberately corrupted backup archive	The verifier rejects it — so we know it checks, rather than always passing.
Send a batch larger than the limit	It returns a clear error, not a silently truncated answer.
Send values that are not real numbers	It returns "bad request", not "server crashed".
Ask for drift on data that has not drifted	It reports stable — no false alarm.
Ask for drift on deliberately shifted data	It reports alert — catches the real thing.
Try to load a runtime that does not exist	A clear, readable message — not a confusing internal error.

WHY THE "NO DRIFT" TEST IS AS IMPORTANT AS THE "DRIFT" TEST

It would be easy to write a drift detector that always shouts "DRIFT!" and pass every test that checks it detects drift. It would also be completely useless.

So the tests check both directions: it must fire on genuinely shifted data, *and* stay quiet on normal data. A detector that fails either one is not usable. Testing only the positive case is one of the most common mistakes in test suites.

One test worth explaining in detail

During the work on this project, the report took 113 seconds to generate because of how the statistics were calculated, and it was made about 10× faster. Whenever you speed something up, you must prove the answer did not change — and "close enough" is not good enough for a replacement that is supposed to be identical.

So there is a test that runs both the old way and the new way **on exactly the same random draws**, and asserts the results match exactly. Not "approximately." Exactly.

That distinction matters. If the test only checked that the answers were close, a genuine difference could hide inside the randomness and nobody would ever know. Pinning the random draws removes the randomness from the comparison entirely, leaving only the thing being tested.

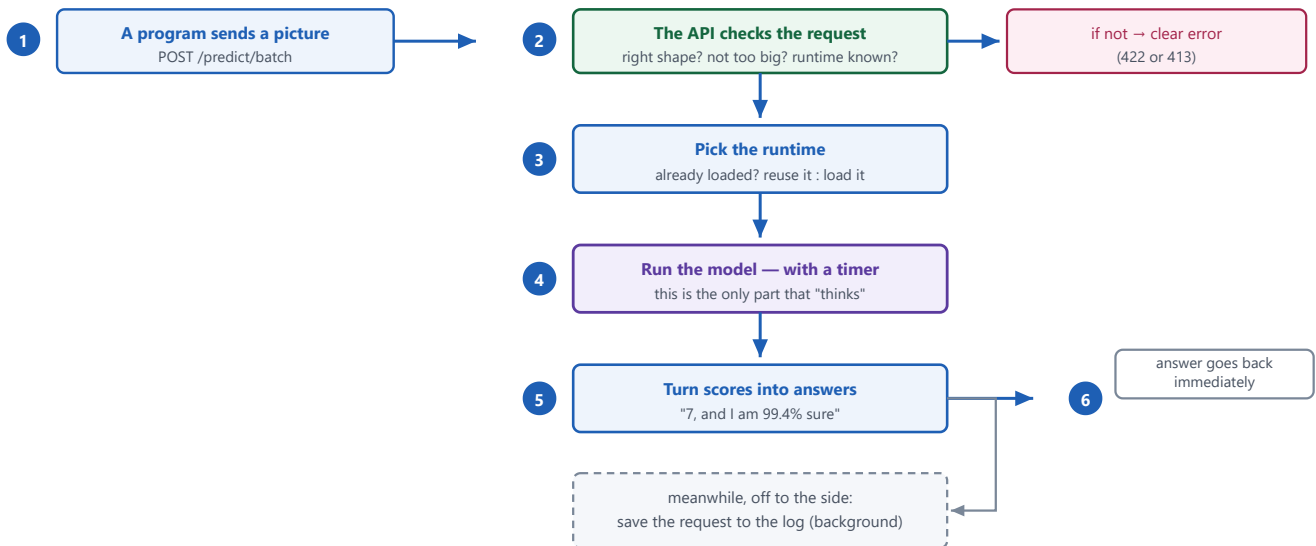
WHERE WILL YOU USE THIS AGAIN?

- **Testing failure paths** is the single most underrated testing habit. It is where the interesting bugs are, and it is what separates a test suite that gives confidence from one that gives the *feeling* of confidence.
- **Testing both directions** — detects when it should, stays quiet when it should — applies to every detector you ever build: alerts, filters, validators, search functions, anomaly detection.
- **Proving an optimisation did not change the result** is how you safely make anything faster. It is also exactly what the export parity check does for the three runtimes.
- **Pin the randomness** in any comparison involving chance, or you are comparing noise to noise.

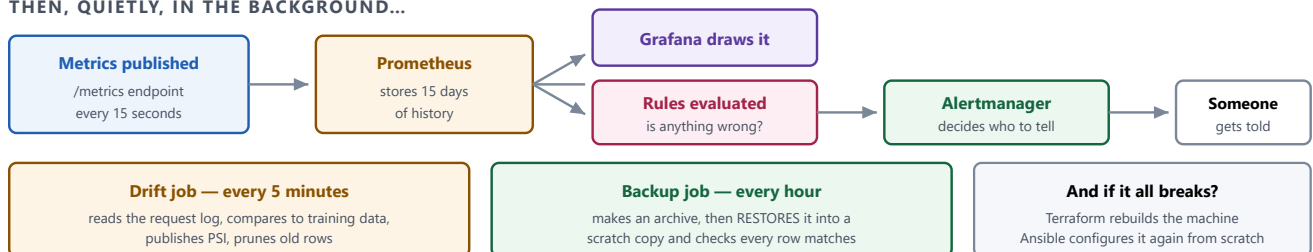
The whole picture

Everything from the previous seventeen parts, on one page, in the order it actually happens.

Follow one request all the way through



THEN, QUIETLY, IN THE BACKGROUND...



One request, six steps, then two background loops that keep the system honest.

The three layers, and why they are separated

LAYER	CONTAINS	WHY IT IS SEPARATE
Knowledge	The model, the reference distribution	Expensive to produce, so it is built once and stored. Everything else consumes it read-only.
Service	The API, the runtimes	Fast, always running, answers users. Replaceable at any time without losing the model.
Governance	Monitoring, drift, backup, tests, CI	Not on the user path at all. These exist so that the other two can be trusted — and they can fail without breaking anything the user sees.

That third layer is the part most projects skip, and it is the entire point of this one. A model plus an API is a demo. A model plus an API plus monitoring, drift detection, verified backups and automated checks is a *service*.

The one-line summary

IF YOU REMEMBER ONE THING

A machine-learning project is **10% model and 90% the engineering that makes the model trustworthy**. This repository is deliberately built so that the 90% is visible — the parts you cannot photograph, demo, or put on a slide, and that determine whether the thing actually works.

Twenty-one mistakes, and what each one teaches

This is the most valuable part of this document. Not because the mistakes are interesting in themselves, but because every one of them is a mistake you will otherwise make yourself — and recognising the *pattern* is what transfers.

These are all real, all found during the work, and all fixed. Some were caught by tests; several were caught only because something looked slightly odd and was investigated properly.

Category 1 — The ones where everything looked fine

#	WHAT WENT WRONG	WHY IT WAS INVISIBLE	THE LESSON
1	The benchmark was not reproducible. Two runs of the identical command timed different inputs.	The difference looked exactly like ordinary timing noise — which is precisely what a benchmark expects to see.	A bug that mimics expected noise is the hardest kind to find, because the output never looks wrong.
2	Python's built-in hashing was used for seeding. Python deliberately randomises it per process.	Nobody suspects the language itself. It is a documented feature, not a bug.	Use an explicitly stable function (like SHA-256) for anything that must be identical across runs. Never assume "the same input gives the same output" unless you have checked.
3	The container's Python was never tested. Deployment moved to 3.14; the checks stopped at 3.13.	Every single check passed. All green. Completely misleading.	A check that passes for the wrong reason is worse than no check. Ask: <i>what would make this go red?</i> For a version matrix, the answer must include the version you actually deploy.
4	The image size in the documentation was wrong — stated as 484 MB when it was 2.24 GB.	It was recorded early and never re-measured. Nobody questions a number they wrote themselves.	Numbers in documentation go stale. Re-measure before publishing, especially numbers that flatter.
5	Drift detection would have been blind to a broken detector. "No drift" and "nothing measured" looked identical.	Both show as "no drift". A dashboard cannot tell them apart — not because the dashboard is bad, but because the information was never recorded.	Always ask of any monitoring signal: <i>what does this look like when the thing producing it is dead?</i>

Category 2 — The ones that only appear in a different environment

#	WHAT WENT WRONG	WHY IT WAS INVISIBLE	THE LESSON
6	Permission denied in the container only. The results folder resolved to a location owned by the administrator.	On a developer's machine that folder is writable. It always worked locally.	"Works on my machine" is usually not carelessness — it is a genuinely different environment. Defaults that resolve relative to a file's location change meaning when the file moves.
7	A storage folder was created owned by the wrong user, so the drift job could not write its output.	The folder was created automatically by the container system, not by our code, so it was not obvious it needed attention.	Create your own directories explicitly, with the right owner, before something else creates them wrongly. Then verify writability during the build so it fails early and loudly.
8	Four of nine containers reported unhealthy while working perfectly. Starting the project appeared to hang forever.	The health check was defined once, in the shared image — but only one of the five jobs using that image actually serves web traffic.	Put a check next to the thing it checks. A check defined in a shared component is a check that assumes all its users are the same shape.
9	Two machines ran different versions of the same quality tool and disagreed about identical files.	Each environment was internally consistent. Only comparing them revealed it.	Pin tool versions. When a check disagrees between environments, treat it as a finding, not an inconvenience. Here it was pointing at a real robustness problem in variable naming.
10	The setup tool refused to read its own configuration file because the directory was world-writable (a Windows-mounted filesystem).	It simply fell back to defaults silently. No error, no warning.	Tools that degrade silently are dangerous. When behaviour differs from expectation, check whether your configuration was read <i>at all</i> .

Category 3 — The ones where the tool lied

#	WHAT WENT WRONG	WHY IT WAS INVISIBLE	THE LESSON
11	The model compression tool produced a file that could not be loaded. It reported success.	Compression <i>had</i> succeeded. The output was produced correctly. It just could not run — which is a different question.	"The tool succeeded" is not evidence the artifact works. Load it and run it before accepting it. The fix was to try candidates and execute each one, keeping the first that actually works.
12	A version number was written without its leading "v", so the security scanner never ran.	It failed during setup, before any step executed — so the failure looked like infrastructure flakiness, not a typo.	Check that your checks are running. A security check that never executes reports exactly the same thing as one that found nothing.

#	WHAT WENT WRONG	WHY IT WAS INVISIBLE	THE LESSON
13	A test alarm endpoint returned "not found" because it did not read the incoming request body first.	The response looked like a routing problem, not a request-handling problem.	With streaming request bodies, you sometimes must consume the body before responding — even if you do not want it. Read the documentation of your own framework as carefully as you read the ones you are learning.

Category 4 — The performance traps

#	WHAT WENT WRONG	WHY IT WAS INVISIBLE	THE LESSON
14	Counting rows after every database write made writes get quadratically slower. The benchmark stalled after a few thousand requests.	Each individual check was fast. The problem only appeared at scale, and looked like a benchmark problem rather than a database problem.	Never put a growing cost on a hot path. Move maintenance work to a background job — which is also why row cleanup lives in the drift job now.
15	Generating the report took 113 seconds. The statistics were computed in a slow loop.	It worked correctly. It was just slow enough that people would stop running it.	A tool that is correct but unpleasant gets abandoned. Speed is a feature when it determines whether something is actually used.
16	A database connection context manager does not close the connection — it only manages the transaction.	The code read naturally and looked like correct idiomatic Python.	A context manager's behaviour is not implied by its appearance. This one leaked a file handle per backup run — invisible on a laptop, quietly fatal on a server running for months.

Category 5 — The statistical traps

#	WHAT WENT WRONG	WHY IT WAS INVISIBLE	THE LESSON
17	The drift detector produced about three false alarms every run. Testing 64 features at 5% significance does that by construction.	Each individual result was statistically valid. The problem was doing 64 of them at once.	Multiple comparisons need correcting. And an alarm that cries wolf gets muted — which is worse than no alarm, because it destroys trust in the whole system.
18	Three features that never vary were included in the drift score , diluting the average and inflating the denominator.	They scored 0, which looks harmless. The damage was in the averages, not in the individual values.	Exclude things that cannot carry information. A feature that never changes cannot detect change — but it can still distort a summary statistic.

#	WHAT WENT WRONG	WHY IT WAS INVISIBLE	THE LESSON
19	Two identical runs gave opposite answers about whether int8 was faster. One said 1.47× slower; the next said 1.75× faster.	Each run was internally consistent and produced a confident-looking number.	The turning point of the project. It led to measuring the harness's own noise — and to the honest conclusion that the harness cannot resolve its own differences. When your instrument is noisier than your signal, the correct output is "I cannot tell", not the biggest number you saw.
20	A percentile calculation was off by one on exact inputs, because 0.1 cannot be represented exactly in binary. Reported "101 needed" instead of "100".	The result was only wrong on round numbers — the exact cases the documentation quoted.	Use integer arithmetic when the answer must be exact. Floating-point is fine for estimates and wrong for thresholds. This was caught by a test that checked the documented examples.
21	Deduplicating bin edges would have broken the reference data — producing a different number of bins per feature.	Deduplication looks obviously correct. It reduces redundancy.	A "cleanup" that makes the data tidier can break the shape your code depends on. Read the comment explaining why the redundant-looking code is there — that comment was written by someone who already made this mistake.

The five patterns behind all twenty-one

1. Silent degradation

Things fall back to defaults, succeed at the wrong thing, or fail where you cannot see. *Defence: check that your checks are running.*

2. Environment divergence

It works here and not there. *Defence: pin versions, and test where you deploy.*

3. Scale-dependent bugs

Fine with ten rows, fatal with a million. *Defence: think about growth, and test at realistic size.*

4. Statistics misapplied

Valid maths, wrong question. *Defence: ask what the number would mean if nothing had changed.*

5. Trusting the tool's own report

"Success" means it finished, not that it worked. *Defence: verify the outcome, not the exit code.*

WHERE WILL YOU USE THIS AGAIN?

Every one of these patterns will find you again, in a different technology, wearing different clothes. The specific bug — a hash, a version tag, a folder permission — will not repeat. The *shape* will, constantly.

The skill that matters is not memorising these twenty-one. It is developing the reflex to ask, whenever something looks fine: **"what would this look like if it were broken?"** That single question is behind most of the fixes above.

Where you will use each of these things

Every tool in this project, and the real-world problems it solves elsewhere. This is the transferable part — what you take with you when you leave this repository behind.

THING	WHAT IT IS REALLY FOR	WHERE YOU WILL MEET IT AGAIN
PyTorch	Building and training models	Research, computer vision, natural language processing, recommendation systems, forecasting. The dominant framework in industry and academia.
ONNX	Moving a model between tools	Running models in browsers, phone apps, C++ and Rust programs, or on specialised hardware. Anywhere your training framework is not available.
int8 compression	Making models smaller and cheaper	On-device machine learning, embedded systems, edge devices, anywhere memory or bandwidth is limited. Also used to reduce cloud inference costs.
FastAPI	Turning code into a web service	Backend APIs, microservices, internal tools, data services. One of the most common Python web frameworks in new projects.
Liveness vs readiness	Letting systems know your state honestly	Kubernetes, Docker Swarm, cloud load balancers, service meshes. Getting this wrong is one of the top causes of production outages.
Benchmarking	Deciding between options with evidence	Database selection, framework choices, hardware sizing, cloud instance types, compiler flags, query tuning. Any decision where "it feels faster" is not good enough.
Confidence intervals	Quantifying uncertainty	A/B testing, capacity planning, load testing, forecast accuracy, survey analysis. Anywhere a single number would be misleading.
Bootstrapping	Uncertainty without assuming a distribution	Latency analysis, financial modelling, biological data, any measurement with a long tail. The modern default because it makes fewer assumptions.
PSI / drift detection	Noticing when reality changes	Fraud detection, credit risk, ad targeting, medical monitoring, demand forecasting, predictive maintenance. Any model deployed into a world that keeps moving.
Verified restore	Knowing your data can actually come back	Every system with data worth keeping. Required by most compliance regimes, and the first thing asked about in any serious operational review.
Checksums	Detecting silent corruption	Package managers, container registries, downloads, backups, data pipelines, version control, filesystem integrity.
Docker	Consistent software environments	Nearly all backend and data work. Local development, testing, deployment, batch jobs, CI.
Docker Compose	Describing a multi-part system	Local development environments, integration testing, small production deployments, demos.

THING	WHAT IT IS REALLY FOR	WHERE YOU WILL MEET IT AGAIN
Prometheus	Storing and querying metrics	Infrastructure and application monitoring at almost any scale. Its query language is a widely-required skill.
Grafana	Visualising what is happening	Every operations team, dashboards for business metrics, incident investigation, capacity planning.
Alertmanager	Deciding who to disturb and when	On-call systems, incident response, escalation policies. The difference between alerting that helps and alerting people turn off.
Ansible	Configuring servers consistently	Server provisioning, configuration management, deployment automation, compliance enforcement across fleets.
Terraform	Creating infrastructure from a description	Cloud provisioning on AWS, Azure, Google Cloud or any of dozens of providers. The standard tool for infrastructure as code.
GitHub Actions	Automating checks and releases	Every professional software team. Testing, building, publishing, deploying, security scanning.
Ruff	Catching style and simple bugs automatically	Python projects of all sizes. Replacing several older tools because it is dramatically faster.
pytest	Proving your code does what you think	Universally used in Python. The skill of <i>what</i> to test matters far more than the framework.
Automated dependency updates	Staying current and secure	Any maintained codebase. Most breaches exploit known vulnerabilities in outdated dependencies, so this is a genuine security control, not housekeeping.

Skills, not tools

Tools change. These do not, and this project exercises all of them:

Reading before writing

Understanding the whole flow before changing anything. The single biggest difference between a fix and a new bug.

Measuring instead of guessing

Reaching for evidence before opinion — and being willing to report that the evidence is inconclusive.

Designing for failure

Asking "what happens when this breaks?" at design time rather than discovery time.

Being honest about limits

Stating what was not tested, what could not be resolved, and what you got wrong. This is rare enough to be a differentiator.

Making the cheap choice deliberately

Not every problem deserves a sophisticated solution. Knowing when something is good enough — and being able to justify it — is senior-level judgement.

Writing for the next person

Comments that explain *why*, not *what*. Documentation that states limitations. The person you are helping is usually future-you.

Glossary

Every term used in this document, in one place. Alphabetical, so you can jump straight to the one you have forgotten.

TERM	PLAIN MEANING
Accuracy	Share of predictions that were correct. 0.9741 means 97.41%.
Alert	An automatic notification that something needs attention.
Alertmanager	The program that decides who to notify when a monitoring rule fires, and stops repeats.
Ansible	A tool that configures computers from a written description, safely repeatable.
API	A fixed set of questions one program agrees to answer for another.
Artifact	A file produced by a build process — here, the exported model files.
Batch	Several items sent together in one request.
Bootstrap	Estimating uncertainty by resampling your existing data many times.
Checksum	A short code computed from a file's contents. If the file changes at all, the code changes. Used to detect corruption.
CI / Continuous Integration	Running automatic checks on every change.
Concurrency	Several things happening at the same time.
Confidence interval	A range that probably contains the true value.
Container	A sealed package of a program plus everything it needs to run.
Drift	When real-world data gradually stops resembling the data a model learned from.
Docker	The most common tool for building and running containers.
Endpoint	One specific address that performs one specific job.
Environment variable	A named setting passed to a program when it starts.
fp32	The standard way to store model numbers: high precision, 4 bytes each.
FastAPI	A Python framework for building web services; generates documentation automatically.
Grafana	A program that draws stored numbers as charts and dashboards.
Histogram	A record of how many values fell into each of several ranges. How latency distributions are stored.

TERM	PLAIN MEANING
HTTP	The language web browsers and services use to talk to each other.
Idempotent	Safe to run repeatedly: doing it twice has the same effect as doing it once.
Inference	Using a trained model to answer a question. The exam, not the studying.
int8	Storing model numbers as small whole numbers: less precise, but a quarter of the size.
JSON	A simple text format for structured data — the standard way services exchange information.
KS test	A statistical test for whether two sets of values differ. Sensitive to sample size, so used here only as supporting evidence.
Latency	How long one request takes.
Liveness	"Is this process still running?" Failure means restart it.
Metric	A number recorded over time so it can be watched.
Model	A program that learned from examples rather than being told rules.
Neural network	A stack of simple maths steps, each passing its result to the next.
Noise floor	The smallest difference an instrument can reliably detect. Measured here by comparing identical things.
ONNX	A common format for models, so different tools can run the same one.
Overfitting	Being excellent at the practice questions and bad at the real exam.
p50 / p95 / p99	Percentiles: the middle request, the slowest 1 in 20, the slowest 1 in 100.
Parity check	Proving two versions of the same thing behave identically.
Percentile	The value below which a given share of observations fall.
Prometheus	A program that collects and stores metrics over time.
PSI	Population Stability Index. A number describing how much a distribution has shifted. Sample-size independent, so it is what decides here.
Quantisation	Storing numbers with less precision to save space. "About 1.4 metres" instead of "1.4273 metres".
Readiness	"Can this serve traffic right now?" Failure means stop sending requests, not restart.
Reproducible	Running it again produces the same result.
Runtime	The program that runs a model — the engine, not the fuel.
Seed	The starting value for a random number generator. Fix it and the randomness repeats exactly.
SQLite	A database that lives entirely in one file. No server to run.

TERM	PLAIN MEANING
Systemd timer	A Linux feature for running a command on a schedule.
Terraform	A tool that creates cloud infrastructure from a written description.
Throughput	How many requests can be handled per second.
TorchScript	A frozen, standalone version of a PyTorch model that does not need your Python code.
Training	Showing a model many examples with correct answers so it adjusts itself.
Variance	How much measurements spread out. High variance means noisy.
Volume (Docker)	A storage area that outlives containers.
Weights / parameters	The adjustable numbers inside a model. Training's entire output.
Wilson interval	A confidence interval for a percentage that stays inside 0–100%, unlike the simpler method.
YAML	A human-friendly text format for settings files.

Command cheat sheet

Everything you can do with this project, in the order you would normally do it. The reason the task runner exists (Part 10) is that these are the only commands you need to remember.

Getting started

```
# Get the code and install what it needs
git clone git@github.com:ziaur390/mlserve-ops.git
cd mlserve-ops
python tasks.py install

# Build the model: train it, export three runtimes, build the drift reference
python tasks.py pipeline
```

Everyday commands

COMMAND	WHAT IT DOES
<code>python tasks.py</code>	List every available command with a description. <i>Start here.</i>
<code>python tasks.py test</code>	Run all 255 checks (fast ones).
<code>python tasks.py test-all</code>	Run everything, including checks that need Docker.
<code>python tasks.py lint</code>	Check code style and catch simple mistakes.
<code>python tasks.py format</code>	Fix style problems automatically.
<code>python tasks.py cov</code>	Run checks and show which lines are not covered.

Working with the model

COMMAND	WHAT IT DOES
<code>python tasks.py train</code>	Train the model from scratch (~4 seconds).
<code>python tasks.py export</code>	Produce the three runtimes, and fail if they disagree.

COMMAND	WHAT IT DOES
<code>python tasks.py reference</code>	Rebuild what "normal input" looks like, for drift detection.
<code>python tasks.py pipeline</code>	All three of the above, in order.
<code>python tasks.py serve</code>	Run the API locally with auto-reload, for development.

Measuring

COMMAND	WHAT IT DOES
<code>python tasks.py bench</code>	Run the full benchmark. Writes <code>results/raw.csv</code> .
<code>python tasks.py report</code>	Build the report and charts from that CSV.
<code>MLSERVE_BENCH_CALLS=10 python tasks.py bench</code>	A quick run, for checking things work.

Operations

COMMAND	WHAT IT DOES
<code>python tasks.py drift</code>	Run drift detection once. Exit code 2 means "drift found" — a finding, not a failure.
<code>python tasks.py seed</code>	Insert synthetic requests, so the backup drill has data.
<code>python tasks.py backup</code>	Create a dated archive with checksums.
<code>python tasks.py restore-verify</code>	Restore the newest archive and prove every row matches.
<code>python tasks.py backup-list</code>	List existing archives.
<code>python tasks.py smoke</code>	Check the running stack actually answers on its endpoints.

The whole stack

COMMAND	WHAT IT DOES
<code>python tasks.py up</code>	Start all nine containers, built and in the right order.

COMMAND	WHAT IT DOES
<code>python tasks.py logs</code>	Follow the logs from everything.
<code>python tasks.py down</code>	Stop everything, keeping the data.
<code>python tasks.py nuke</code>	Stop everything and delete the data . Destructive.

The one command that checks everything

```
# Style, tests, model build, benchmark, report, drift, backup, restore-verify
python tasks.py verify

# The same, plus the full container stack and a live smoke test (slow, needs Docker)
python tasks.py verify-all
```

Where things live

PATH	WHAT IS IN IT
<code>src/mlserve/</code>	All the code.
<code>src/mlserve/bench/</code>	The benchmark: runner, statistics, report.
<code>src/mlserve/drift/</code>	Drift reference and detection.
<code>src/mlserve/ops/</code>	Backup, restore verification, test data.
<code>configs/</code>	All the settings, with reasons in the comments.
<code>artifacts/</code>	Generated: model files, drift reference.
<code>results/</code>	Generated: raw data, report, charts.
<code>backups/</code>	Generated: backup archives.
<code>tests/</code>	All 255 checks.
<code>monitoring/</code>	Prometheus, Alertmanager and Grafana configuration.
<code>ansible/</code>	Server setup description, 6 roles.
<code>terraform/</code>	Infrastructure description, 2 environments.
<code>.github/workflows/</code>	The three automatic checking jobs.

PATH	WHAT IS IN IT
<code>VERIFICATION.md</code>	The record of what was proven, and what was not.

Web addresses when the stack is running

ADDRESS	WHAT YOU WILL SEE
<code>http://localhost:8000/docs</code>	A clickable page describing every API endpoint. Try it here.
<code>http://localhost:8000/metrics</code>	The raw numbers being published.
<code>http://localhost:3000</code>	Grafana — the dashboards.
<code>http://localhost:9090</code>	Prometheus — query the stored metrics.
<code>http://localhost:9093</code>	Alertmanager — see what alerts are active.

ONE LAST THING

If you take a single habit away from this document, take this one:

When something works, ask what would make it fail. When something fails, ask why it was invisible. And when you cannot tell, say so — clearly, and with the reason.

That is what separates this project from a demo, and it is the whole difference between writing code that runs and building systems people can rely on.